VINUNIVERSITY

Fine-tuning LLMs — Từ Full Fine-tune đến LoRA/QLoRA

AICB-P2T3 · Ngày 21 · Chương 5 — Fine-tuning & An Toàn

Giảng viên

VinUniversity · Phase 2 · Track 3 · Tuần 5

HÃY SUY NGHĨ...

“Bạn có 5.000 hội thoại chăm sóc khách hàng bằng tiếng Việt. Prompt engineering đã hết đường — bây giờ làm gì?”

Giữ câu hỏi này trong đầu khi học bài hôm nay

Nội dung bài học — 5 phần, 25 module

I Quyết định

1. Có nên fine-tune không?

II Model đến từ đâu

2. Vòng đời model & các lab frontier
3. Pre-training
4. Mid-training
5. Post-training
6. CPT vs SFT vs Distillation
7. Optimizer & kiến trúc

III Lõi fine-tuning

8. Full fine-tune & bộ nhớ
9. Bản đồ PEFT
10. LoRA — cơ chế
11. LoRA Without Regret
12. Biến thể LoRA
13. QLoRA & lượng tử hoá
14. Siêu tham số & vòng lặp
15. Tăng tốc: kernel, loss
16. Công cụ & dịch vụ

IV Dữ liệu

17. Dataset, template & masking
18. Dữ liệu tổng hợp
19. Dữ liệu trajectory (agent)
20. Data engine & dữ liệu chuyên gia

V Ra sản phẩm

21. Đánh giá
22. Thất bại & debug
23. Merge, deploy & serving
24. License & pháp lý
25. Demo & Lab 21

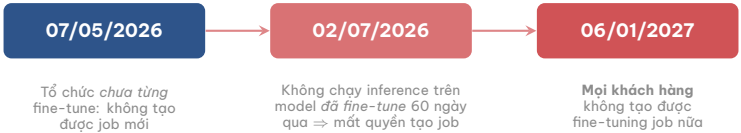
Khi nào cần Fine-tune?

01

Prompt engineering đã đủ chưa — và ai còn fine-tune hộ bạn?

Bối cảnh 2026 — OpenAI thu hẹp fine-tuning self-serve

OpenAI thông báo ngừng nền tảng self-serve fine-tuning



Điều gì vẫn còn

Inference trên model đã fine-tune chạy **đến khi base model bị khai tử** — và `o4-mini` (đường RFT) tắt ngày **23/10/2026**. Managed FT *chưa* chết: Vertex (Gemini SFT; RFT preview *chỉ để thử nghiệm*), Bedrock (RFT cho gpt-oss, Qwen3), Together, Fireworks (§16).

Lưu ý: Không flagship nào ra mắt 09/2026 (Claude 5.5, GPT-6.1 Sol, Gemini 4 Argon, Grok 4.7) cho khách fine-tune. **Tự fine-tune model mở** mới là con đường bền vững — nội dung hôm nay (§2).

Nguồn: developers.openai.com/api/docs/deprecations (kiểm 06/10/2026); trang model gpt-6.1-sol: Fine-tuning "Not supported".

Fine-tune sửa được gì — và KHÔNG sửa được gì

VINUNIVERSITY

Sửa tốt — Hành vi

- Định dạng đầu ra cố định
- Giọng văn, phong cách
- Thuật ngữ chuyên ngành
- Tuân thủ schema/JSON
- Rút ngắn prompt ⇒ giảm token

Kém — Tri thức mới

- Sự kiện thay đổi liên tục
- Dữ kiện hiếm, cần trích dẫn

⇒ **RAG** (Ngày 3–4).

Nhưng SFT *có* dạy được tri thức hẹp, ổn định (bằng chứng 2026 — ghi chú).

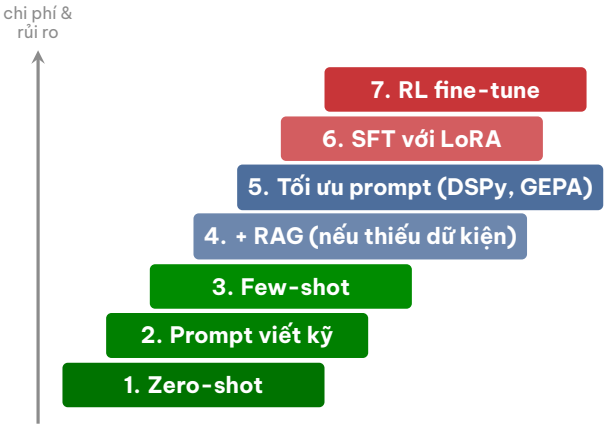
Sửa được — Kinh tế

- Model 8B fine-tuned thay model 200B + prompt dài
- Latency thấp hơn
- Chạy on-prem (dữ liệu không rời tổ chức)
- Không phụ thuộc vendor

Kiểm tra một câu

Câu trả lời đúng *thay đổi theo thời gian* ⇒ RAG. *Cách trả lời* là vấn đề ⇒ fine-tune. Cả hai ⇒ fine-tune định hình hành vi, RAG cấp dữ kiện. “Fine-tune chỉ dạy hành vi, không dạy tri thức” là **nói quá**.

Bậc thang baseline — leo từng bậc, đo ở mỗi bậc



Mỗi bậc *phù hợp với bài toán* là một **baseline cần đo** trước khi bước lên bậc trên. Chỉ bậc 6–7 (**đỏ**) mới đổi trọng số.

Bậc 5 mạnh hơn bạn nghĩ: GEPA — tối ưu prompt bằng cách đọc lại trajectory lỗi — thắng RL (GRPO) **6% trung bình**, tới 20%, và dùng rollout ít hơn **tới 35 lần** — trên 6 tác vụ được nghiên cứu.

Không phải lối đi bắt buộc

Dữ kiện hay đổi thì RAG lên sớm. Thiếu năng lực chung thì thử model *lớn hơn* trước khi fine-tune model nhỏ.

Cây quyết định 2026 — sáu câu hỏi và một cổng



Từ khoá: **SFT** học từ câu trả lời mẫu · **LoRA** chỉ cập nhật adapter nhỏ · **RL** học từ tín hiệu thưởng · **verifier** bộ kiểm kết quả tự động · **bộ eval** = metric, holdout, ngưỡng hồi quy, chi phí/độ trễ — chốt *trước* (§21).

Khi nào fine-tuning có lãi? — tính thử một lần

Bài toán: 1 triệu request. **A:** Gemini 3.8 Flash + prompt dài. **B:** Gemini 3.1 Flash-Lite đã tune, prompt ngắn. *Giả định* cùng chất lượng.

	A: prompt	B: tuned
Token vào/ra mỗi request	1.600/200	300/200
Giá vào/ra ($/1M token)	0,75/3,75	0,375/2,25
Chi phí / 1M request	**$1.950**	**$562,50**

Tiết kiệm **$1.387,50** mỗi triệu request. Train 500 mẫu × 500 token × 3 epoch = 750K token = **$2,25**. Tiền train gần như bằng 0 — *tiền thật nằm ở dữ liệu và đánh giá.*

```
hoa_von = F / (cost_A − cost_B)
```

F: dữ liệu, eval, người	Hoà vốn
$500	360K request
$5.000	3,6M request
$5.000, giá 2027	1,5M request

Lưu ý: Đáp án đổi 10 lần theo **F** — con số bạn hay đoán sai nhất. Và endpoint tuned của Gemini tính **gấp 1,5** giá base.

Fine-tune thay đổi *nhiều hơn* thứ bạn nhắm tới

Hành vi bên ngoài tác vụ

- Fine-tune trên dữ liệu *lành tính* vẫn có thể làm yếu căn chỉnh an toàn (2023).
- **Emergent misalignment:** train hẹp vào "viết code không an toàn" ⇒ model lệch hành vi *rộng*.
- Hai prompt train *cùng nghĩa*, cùng điểm — nhưng mức quên tác vụ khác lệch hẳn (2026).
- SFT có thể **xoá** khả năng suy luận dù accuracy vẫn tăng (§17).

Vòng đời và phụ thuộc

- Adapter **đi kèm base model** — base bị khai tử, adapter chết theo.
- Bedrock: model chuyển sang Legacy thì **không tạo job fine-tune mới** được nữa.
- License của base đi theo bản fine-tune (§24).
- Tuned ≠ riêng tư: Gemini tuned chỉ chạy trên endpoint công cộng dùng chung.

Hệ quả bắt buộc: một bộ hồi quy đi cùng mọi lần fine-tune

Đo **ba thứ**: điểm tác vụ, điểm tác vụ *khác* (năng lực chung), hành vi an toàn. Lab 21 đo target, regression, format (§21).

Có nên fine-tune không?

Ý tưởng: Fine-tune là bậc thang cuối, không phải nút đầu tiên — và phải chứng minh nó thắng.

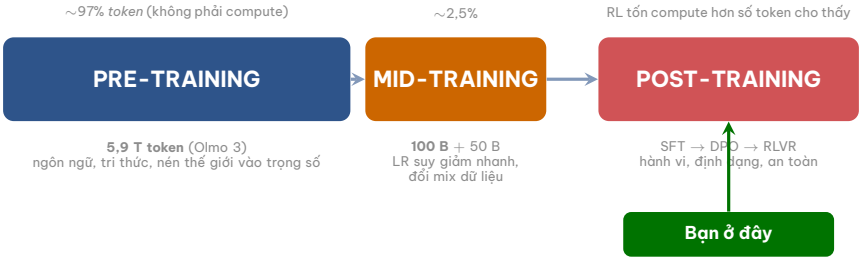
●○○ **Cơ bản**	Thử prompt tốt trước, đo kết quả. Chỉ fine-tune phần còn thiếu, khi việc đó lặp lại nhiều và bạn có ví dụ mẫu.
●●○ **Nâng cao**	Leo bậc thang baseline; dữ kiện hay đổi thì RAG; tính hoà vốn với chi phí dựng *thật*; mọi lần tune đi kèm bộ hồi quy và an toàn.
●●● **Chuyên sâu**	So với tối ưu prompt (GEPA) trước khi chọn RL; tách tri thức ổn định (SFT dạy được) với tri thức hay đổi; quản lý vòng đời base model, license và endpoint như một phụ thuộc phần mềm.

Vòng đời một mô hình 2026

Pretrain → Mid-train → Post-train — và bạn bước vào ở đâu

02

Bản đồ ba giai đoạn — và vị trí của bạn



Vì sao bản đồ này quan trọng với người fine-tune

Fine-tuning **không phải một kỹ thuật riêng biệt** — nó là *giai đoạn cuối cùng của cùng một đường ống*, chạy trên ngân sách nhỏ hơn hàng nghìn lần. Mọi kỹ thuật ở §5–§6 (SFT, distillation, RL) đều là thứ các lab đã chạy trước bạn, chỉ khác quy mô. Hiểu hai giai đoạn đầu để biết mình *thừa hưởng* gì — và cái gì đã *không còn sửa được* bằng fine-tune.

“Model flow” — vì sao Olmo 3 là ví dụ dạy học tốt nhất

Hầu hết các lab chỉ công bố **trọng số cuối**. Ai2 công bố *toàn bộ* **model flow** — định nghĩa của chính họ:

> **Model flow** — “Toàn bộ vòng đời của một mô hình ngôn ngữ: *mọi* giai đoạn, checkpoint, tập dữ liệu và phụ thuộc cần để tạo ra và sửa đổi nó.”

Nghĩa là bạn tải được checkpoint *giữa* pre-training, *sau* mid-training, *trước* RL — và đo xem mỗi giai đoạn thực sự thêm năng lực gì.

Dolma 3 (Olmo 3)	Token
Toàn bộ kho ngữ liệu	9,3 T
Mix — pre-training	5,9 T
Dolmino — mid-training	100 B
Longmino — ngữ cảnh dài	50 B

Dolmino rút từ bể 2,2 T; Longmino rút từ bể 639 B. Mô hình: Base / Think / Instruct / RL-Zero, 7B & 32B.

Con số đáng nhớ

Mid-training lõi (Dolmino) chiếm ≈**1,6%** ngân sách token, cộng ngữ cảnh dài là ≈2,5% — nhưng là nơi dồn dữ liệu toán, code, suy luận chất lượng cao. *Tỷ lệ dữ liệu tốt quan trọng hơn tổng khối lượng* — bài học lặp lại ở §17.

Model mở 10/2026 — cỡ bạn thật sự fine-tune được

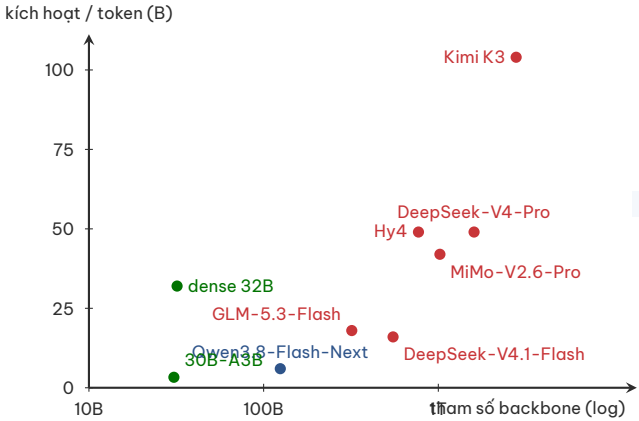
Model (ngày)	Tổng / kích hoạt	License	Ghi chú cho người fine-tune
Qwen3.5-4B (02/2026)	4B dense	Apache 2.0	**model của Lab 21**; đa phương thức
Qwen3.8-27B (08/2026)	27B dense	Apache 2.0	dense, dễ tune nhất ở cỡ này
Gemma 4 (04/2026)	dense $\leq$31B; 26B-A4B	Apache 2.0	Google: "tune and deploy" thương mại
Muse Glimmer (08/2026)	30B dense	Apache 2.0	Meta; agentic; SFT + OPD + RL
Granite 4.2 (08/2026)	3B, 8B, 30B (dense)	Apache 2.0	gọi hàm kiểu OpenAI; RL sandbox
Nemotron 3.5 Lightning (08/2026)	30B / 3B	OpenMDW 1.1	hybrid Mamba-2; recipe LoRA & RL
SEA-LION v4.8 (09/2026)	30B / 3B	MIT	**có tiếng Việt**; *chưa căn chỉnh an toàn*
gpt-oss-20b (08/2025)	21B / 3,6B	Apache 2.0	OpenAI; Bedrock RFT hỗ trợ
Olmo 3 (11/2025)	7B, 32B (dense)	Apache 2.0	mở cả dữ liệu & checkpoint

Lưu ý: "Open weights" $\neq$ Apache: Qwen3.8-Flash-Next, Kimi K3, MiniMax M3 có license *riêng*; Mistral Medium 3.5 trên $20 triệu/tháng cần giấy phép thương mại (§24).

Chọn base theo thứ tự này

License → ngôn ngữ (tiếng Việt?) → cỡ vừa GPU (§13) → dense trước MoE → chat template & tool format.

“A3B” không có nghĩa là nhẹ — tổng vs kích hoạt



Kích hoạt = số tham số *tính toán* cho mỗi token. Nhưng **mọi** expert vẫn phải nằm trong bộ nhớ.

Sàn bộ nhớ, r=16	LoRA bf16	QLoRA
Qwen3-32B dense	63 GiB	19,9 GiB
Qwen3-30B-A3B	69 GiB	28,1 GiB

Model “3B kích hoạt” cần *nhiều* bộ nhớ hơn model dense 32B — vì adapter `all-linear` gắn vào **từng** expert (844M tham số adapter, so với 13M nếu chỉ gắn attention).

Lab đóng — họ kể gì về cách train, và cho bạn tune gì

Lab	Công bố về training	Chữ ký post-training	Khách fine-tune được (10/2026)	Trọng số mở
Anthropic	pretrain rộng; "substantial post-training" theo constitution	căn chỉnh theo constitution	không tìm thấy đường public cho Claude 5.5	—
OpenAI	thành phần dữ liệu pretrain + chấm điểm trong RL	RL grading, deliberative alignment	self-serve thu hẹp (§1); Azure: GPT-4.1, o4-mini RFT	gpt-oss
Google	model card thưa; tài liệu tuning chi tiết	—	Gemini 3.5 Flash / 3.1 Flash-Lite: SFT; RFT preview *không cho production*	Gemma 4
Meta	Muse Spark: scaling + RL; Glimmer: 3 giai đoạn rõ	SFT + OPD + RL	chưa có endpoint tune	Muse Glimmer
xAI	Grok 4.7: RL dài hơn, task khó hơn	RL trên task nhiều giờ	không có dịch vụ public	Grok 2, Grok 1
Mistral	sản phẩm; Forge cho doanh nghiệp	Forge: pretrain → RL	FT host cũ **đã deprecated**; Forge	Small 4
Microsoft	Phi: lọc, sửa lỗi, dữ liệu tổng hợp	mode token suy luận/không	Foundry: Phi-4, Phi-4-mini	Phi-4 (MIT)
Amazon	Nova Forge mở checkpoint từng giai đoạn	SFT, RLHF (Nova 1)	Bedrock: Nova 2 Lite SFT + RFT	—

Nhiều API flagship chưa cho fine-tune trực tiếp

Kiểm catalog từng dịch vụ tại ngày dùng. Mang về từ lab đóng: **phương pháp** (constitution, RL task dài, mode token).

Pre-training — cái bạn thừa hưởng

Bạn không chạy giai đoạn này. Nhưng nó quyết định trần của mọi thứ bạn làm sau đó

Sáu thứ bạn thừa hưởng — và fine-tune không sửa được

Thừa hưởng	Fine-tune sửa được?
Tokenizer & vocab	× Gần như không
Kiến trúc, số lớp	× Không
Độ dài ngữ cảnh gốc	~ Chỉ bằng CPT tốn kém
Ngôn ngữ chưa được học	~ Rất tốn (§6)
Bias ẩn của optimizer	~ Xem §7
Hành vi, định dạng	✓ **Việc của bạn**

Quy tắc chọn base model

Chọn base theo thứ tự:

1. Đã thấy đủ *tiếng Việt* chưa? — đo perplexity trên văn bản miền của bạn *trước* khi huấn luyện.

2. License cho phép mục đích của bạn không (§24)?

3. Kích thước có phục vụ nổi không (Ngày 20)?

Chỉ sau đó mới xét benchmark.

Một câu để nhớ

Fine-tuning **định hình lại** thứ mô hình đã biết, không **tạo ra** năng lực chưa có. SFT dạy được tri thức *hẹp*; 5.000 mẫu không biến base chưa thấy tiếng Việt y khoa thành giỏi *y học*.

Dữ liệu pre-training — từ "lọc bỏ 90%" sang "viết lại"

VINUNIVERSITY

Thế hệ 2024 — lọc quyết liệt

FineWeb-Edu, **DCLM**: dùng classifier chấm điểm chất lượng, giữ lại phần đầu bảng.

Kết quả: điểm benchmark tăng mạnh — nhưng **loại bỏ ∼90% dữ liệu**.

Vấn đề: tài liệu "kém" thường chỉ là *trình bày* kém, không phải *nội dung* kém.

Thế hệ 2026 — tổng hợp lại

Nemotron-CC: kết hợp *nhiều* classifier + **viết lại bằng LLM** thay vì vứt bỏ.

Tài liệu kém ⇒ **viết lại**. Tài liệu tốt ⇒ **nhân bản** thành QA, trích xuất tri thức, dữ liệu distillation.

Mục tiêu đổi từ *độ tinh khiết* sang **sản lượng token hữu ích**.

Chi tiết đáng chú ý

Bản làm mới của Nemotron-CC dùng chính **Qwen3-30B-A3B** làm mô hình viết lại — *một model mở đang là dây chuyền dữ liệu cho model mở kế tiếp*. Cùng một logic áp dụng ở quy mô của bạn tại §17: dọn dẹp và viết lại dữ liệu bẩn thường lãi hơn là vứt nó đi.

Scaling laws 2026 — vì sao base model đã bão hoà

Chinchilla (2022) hỏi: *ngân sách FLOPs cố định, chia bao nhiêu cho tham số, bao nhiêu cho token?* Năm 2026 hàm chi phí đã đổi — và câu trả lời đổi theo.

1. Chi phí suy luận. Model được gọi hàng triệu lần ⇒ huấn luyện thêm cho một model *nhỏ* rẻ hơn phục vụ một model *lớn*. Tính cả việc lấy mẫu nhiều lần lúc suy luận, "huấn luyện quá tay" thành **tối ưu**.

2. Dữ liệu là ràng buộc, không phải compute. Qua một ngưỡng, **lặp dữ liệu hết lãi** — phải tăng năng lực model. Weight decay tối ưu khi thiếu dữ liệu có thể **lớn hơn một bậc**.

Lưu ý: **Hệ quả cho bạn.** Base 7–30 B năm 2026 đã ăn hàng chục nghìn tỷ token — nó **đã bão hoà** dữ liệu phổ thông.

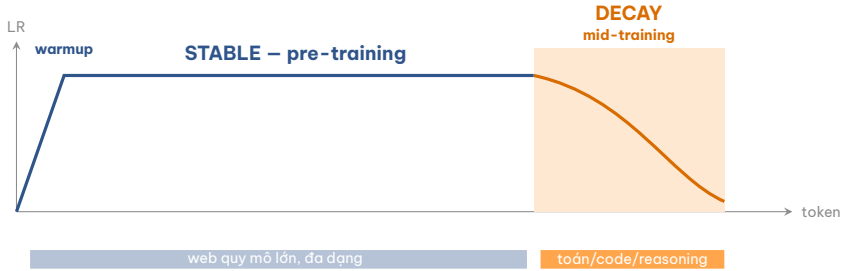
Nên 50.000 mẫu "chất lượng khá" cào từ web thường **không dịch chuyển gì**.

Dữ liệu fine-tune chỉ có giá khi **mới về phân phối**: dữ liệu nội bộ, nhãn chuyên gia, định dạng riêng của bạn.

Mid-training — giai đoạn ít ai gọi tên

Chỉ ≈1,6% ngân sách token, nhưng là nơi dồn dữ liệu chất lượng cao nhất

Định nghĩa & vị trí — đọc được đường learning rate



Hai thứ đổi *cùng lúc*: learning rate suy giảm nhanh **và** hỗn hợp dữ liệu chuyển sang chất lượng cao.

Mid-training — Giai đoạn *bắc cầu* giữa pre-training và post-training: dùng dữ liệu và compute trung gian để nâng năng lực cụ thể (toán, code, suy luận, ngữ cảnh dài) **trong khi vẫn giữ** năng lực nền. Thuật ngữ chưa thống nhất trong tài liệu — hãy định nghĩa trước khi dùng.

Mid-training mua được ba thứ

1. Năng lực

Tăng tỷ trọng STEM, code, và **reasoning traces**.

Olmo 3 *Dolmino*: **100 B** token rút từ bể 2,2 T.

2. Ngữ cảnh dài

Tài liệu dài + nới tần số RoPE.

Olmo 3 *Longmino*: ∼**50 B** token rút từ bể 639 B.

3. Sẵn sàng cho RL

Vì mix đã chứa reasoning traces, **RL chạy thẳng trên base** được — không cần SFT trước.

Cùng một công thức xuất hiện ở model đóng: Meta mô tả mid-training của **Muse Glimmer** là "dữ liệu ngữ cảnh dài hơn, thiên về agent, với nhiều reasoning traces hơn" — đặt giữa pre-training bằng logit distillation và post-training SFT + on-policy distillation + RL.

Ranh giới với §6

Continued pre-training (CPT) ở §6 chính là cỗ máy này thu nhỏ về ngân sách của bạn: đổi mix dữ liệu, LR nhỏ, giữ năng lực nền. Khác biệt là *quy mô* — 100 B token so với 100 M — chứ không phải bản chất.

Mở rộng ngữ cảnh — rẻ để chạy, đắt để chạy *đúng*

Cơ chế: RoPE mã hoá vị trí bằng tần số quay. Muốn dài hơn thì *nới* phổ tần số đó.

Cách	Ý tưởng
Nội suy vị trí	Nén thẳng chỉ số vị trí
NTK-aware	Chiều tần số cao nới ít hơn
YaRN	NTK-by-parts + chỉnh nhiệt độ
LongRoPE	Tìm hệ số nới theo từng chiều

Lưu ý: **Khoảng cách hai con số này là cả bài học:**

YaRN: 128 K bằng **200 bước** từ checkpoint 64 K.

Llama 3.1: 8 K → 128 K bằng **sáu giai đoạn**, ∼**800 B token**.

Phẫu thuật RoPE cho cửa sổ *chạy được*; cửa sổ *dùng được thật* tốn ngân sách mid-training thật.

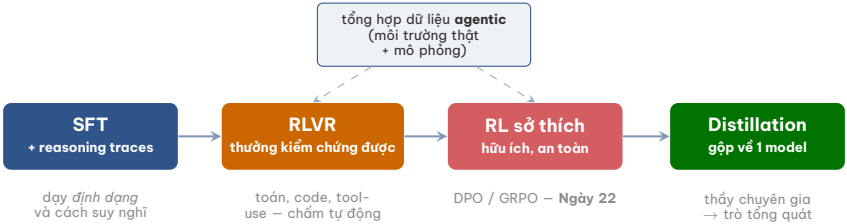
Ảnh hưởng tới bạn

Model card ghi "1 M": *đã huấn luyện tới đó, hay chỉ ngoại suy?* Fine-tune ở 2 K rồi dùng ở 200 K là dựa vào vùng dữ liệu của bạn chưa chạm.

Post-training — công thức của các lab

Cùng một bộ xương lặp lại ở DeepSeek, Kimi, Qwen, Gemma, Muse

Bộ xương chung — và bạn đang làm bước nào



Bạn ở bước 1 — và đó là bước đúng

Với gần như mọi dự án doanh nghiệp, **SFT là toàn bộ công việc**. Ba bước sau cần hạ tầng rollout, hàm thưởng kiểm chứng được và ngân sách GPU của một lab. Biết chúng tồn tại để *đọc được tech report* và để biết khi nào mình đã chạm trần của SFT.

Bốn công thức thật — đọc ngang để thấy điểm chung

Model	Công thức post-training đã công bố
Kimi K2	Đường ống **tổng hợp dữ liệu agentic quy mô lớn** + một giai đoạn **RL hợp nhất**, học qua tương tác với môi trường *thật và mô phỏng*.
Qwen3	**Bốn giai đoạn**: hai giai đoạn xây năng lực *thinking*, hai giai đoạn **hợp nhất chế độ** (`/think` + `/no_think` + ngân sách suy nghĩ). Model nhỏ sinh ra bằng **strong-to-weak distillation**: off-policy trước, on-policy sau.
DeepSeek-V4	**Nuôi chuyên gia theo miền** (mỗi miền một model: SFT rồi RL) → **gộp về một model** bằng **on-policy distillation**.
Muse Glimmer	Pre-train bằng **logit distillation** từ Muse Spark → mid-train agentic → post-train **SFT** + **on-policy distillation** + **RL**.

Điểm chung

Ba trong bốn có bước **distillation**; Kimi K2 kết thúc bằng RL hợp nhất — *không có thứ tự chung*. Model bán ra thường là *học trò* của một *thầy* đắt hơn (cơ chế: §6).

“SFT mở rộng, RL siết lại” — và bảng tên để đọc tech report

Hai vai trò khác nhau — **SFT mở rộng** — thêm hành vi, định dạng, cách diễn đạt mà model *chưa* có.

RL siết lại — làm nhọn phân phối quanh thứ model *đã* làm được.

Hệ quả thực dụng: RL không dạy được kiến thức mới cho model. Nếu năng lực chưa có, **SFT trước — luôn luôn**.

Tên	Sửa vấn đề gì của GRPO
DAPO	Lấy mẫu động + clip bất đối xứng, giữ tín hiệu học
GSPO	Tỷ lệ theo *chuỗi* thay vì theo token — ổn định RL trên **MoE**
CISPO	Clip *trọng số* importance sampling thay vì clip cập nhật token (MiniMax-M1)
Rollout bất đồng bộ	RL đồng bộ vỡ trận với tác vụ agentic dài

Ranh giới rõ ràng

Ngày 21 dạy **công thức**; **Ngày 22** dạy **thuật toán** — DPO, ORPO, GRPO, RLVR, hàm thưởng, reward model. Bảng trên chỉ để bạn không khựng lại khi gặp các chữ viết tắt này trong một tech report.

06

CPT vs SFT vs Distillation

SFT không phải công cụ huấn luyện duy nhất bạn có

Ba công cụ, ba loại ngân sách

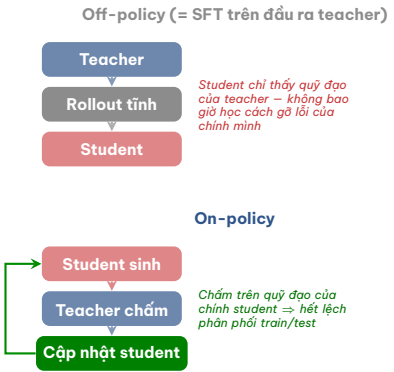
	Continued Pretraining	SFT	Distillation
Dạy điều gì	Tri thức, phân phối miền	Hành vi, định dạng	Bắt chước một teacher
Dữ liệu	Văn bản thô, không nhãn	Cặp có nhãn	Đầu ra của teacher
Đơn vị đo	**Token** (tỷ)	**Mẫu** (nghìn)	Lượt rollout
Mục tiêu	Next-token	Next-token có mask	Khớp phân phối teacher
Rủi ro chính	Quên thảm hoạ	Quá khớp	Lệch phơi nhiễm

Chúng nối tiếp nhau

Một checkpoint đã qua CPT là **điểm xuất phát tốt hơn** cho SFT: SFT theo miền hội tụ nhanh hơn và đạt chất lượng cao hơn khi khởi đầu từ CPT.

Lưu ý: Nếu vấn đề của bạn là "model không *biết*" thì vài nghìn mẫu SFT hiếm khi đủ cho một lỗ hổng *rộng* — so CPT và RAG bằng eval trước khi đổ thêm mẫu.

Distillation — và vì sao "on-policy" quan trọng



Lệch phơi nhiễm — Student học trên trạng thái *teacher* tạo ra, nhưng khi chạy thật phải xử lý trạng thái *do chính nó* tạo ra — gồm cả sai lầm teacher không bao giờ mắc.

Công thức hai giai đoạn

1. SFT trên rollout teacher — kéo student lại gần
2. Tiếp tục bằng on-policy distillation

Nguồn: Thinking Machines Lab, *On-Policy Distillation* (2025)

Quên thảm hoạ — và liều thuốc rẻ tiền

Quên thảm hoạ — Model học tốt miền mới nhưng **đánh mất năng lực tổng quát** đã có — toán, đa ngôn ngữ, tuân thủ chỉ dẫn.

Năm họ giải pháp:

- Chính quy hoá — phạt việc trôi khỏi trọng số quan trọng
- **Replay** — trộn lại dữ liệu phân phối gốc
- Model merging — nội suy giữa base và bản fine-tune
- Kiến trúc — **LoRA chính là họ này**: base đóng băng *là* bản sao được bảo toàn
- Chiếu trực giao — OPLoRA và tương tự

Tỷ lệ replay

1–5% dữ liệu gốc đã giảm quên rõ rệt với chi phí downstream không đáng kể.

Với continued pretraining, khoảng **5–20%** thường được dùng.

Ngay cả **1%** cũng đo được tác dụng.

Luật vận hành

Đo benchmark tổng quát *trước và sau*. Tụt **3–5 điểm** ⇒ tăng replay, hoặc chuyển sang LoRA.

07

Optimizer & Kiến trúc 2026

Muon, MoE thưa, attention lai — và điều gì trong đó thực sự chạm tới bạn

Muon — optimizer đầu tiên soán ngôi Adam ở quy mô frontier

Ý tưởng một câu — Adam chuẩn hoá **từng phần tử** — nó không biết ma trận trọng số là ma trận. Muon coi momentum của một ma trận *là ma trận*, rồi **trực giao hoá** nó (cân bằng mọi trị kỳ dị) trước khi cập nhật.

Hệ quả: những hướng cập nhật *hiếm nhưng quan trọng* không còn bị các hướng trội nhấn chìm.

Cái giá: phép trực giao hoá dùng lặp **Newton–Schulz**, cần nhân ma trận đặc — và nó *xung đột* với trọng số đã bị chia mảnh (FSDP/TP). **Dion** giải quyết bằng xấp xỉ hạng thấp + QR/Cholesky trên ma trận nhỏ, chạy được với DDP/FSDP/TP mà không cần gom lại ma trận đầy đủ.

Model	Optimizer
Kimi K2 (1 T)	**MuonClip**
DeepSeek-V4 (1,6 T)	**Muon**
GLM-5 (744 B)	Muon *(báo cáo)*

MuonClip = Muon + QK-Clip

Khi attention logits bùng nổ, **QK-Clip** co thẳng phép chiếu query/key. Kết quả Kimi K2: **15,5 T token, không một lần loss spike** ở quy mô nghìn tỷ tham số.

Đây là ví dụ sạch nhất của *đổi optimizer là quyết định hạ tầng, không phải tinh chỉnh*.

Kiểm chứng — “nhanh gấp đôi” còn lại bao nhiêu?

Một nghiên cứu đối chứng so **11 optimizer** với *cùng mức dò siêu tham số cho từng cái*. Kết quả nên đọc trước khi tin bất kỳ con số tăng tốc nào:

Điều thường được nói	Điều đo được khi dò công bằng
“Muon nhanh gấp 2× AdamW”	$\leq$**1,4**× ở 0,1–0,5 B; $\approx$**1,1**× ở 1,2 B
“Optimizer ma trận luôn thắng”	Đúng so với loại vô hướng — nhưng ở tỷ lệ dữ liệu 16× Chinchilla, **Soap thắng Muon**
Baseline là công bằng	**Chỉ dò một siêu tham số** trong công thức GPT-3 đã cho riêng AdamW **2**×
Dùng chung siêu tham số là ổn	Weight decay tối ưu: **Lion** $\approx$**0,6** vs **AdamW** $\approx$**0,1**
Nhìn loss sớm là biết ai thắng	**Thứ hạng đảo chiều** khi LR suy giảm — chỉ so ở *cuối* run

Cách hoà giải hai sự thật

Nghiên cứu trên đo model $\leq$1,2 B. Các lab dùng Muon chạy lớn hơn thế **100–1000**×, và lý do họ nêu là **ổn định** + **hiệu quả token** ở quy mô nghìn tỷ — không phải con số 2× trên blog. Cả hai đều đúng, ở hai chế độ khác nhau.

Optimizer có chạm tới bạn không?

Lưu ý: **Đừng đổi optimizer vì thấy nó trong tech report.**

Chuyển thẳng sang Muon để fine-tune một model *đã pre-train bằng Adam* làm **giảm** chất lượng — gọi là **lệch optimizer**: Adam và Muon có *thiên lệch ẩn* khác nhau, và việc chuyển làm **xáo trộn tri thức đã học**.

Mức độ hư hại **tỷ lệ với độ lớn của bước cập nhật**.

Và đó chính là lý do LoRA cứu được

Vì LoRA **giới hạn độ lớn cập nhật**, nó thu hẹp khoảng cách Adam–Muon vốn thấy ở full fine-tune — trên cả tác vụ ngôn ngữ lẫn thị giác.

Nếu vẫn muốn dùng Muon để fine-tune:

- **dùng LoRA**, đừng full FT
- **đừng bê learning rate từ Adam sang**
- dò lại rank — biến thể LoRA chỉnh cho Adam không tự động đúng với Muon

Mặc định vẫn không đổi

Mặc định: **AdamW** / `adamw_8bit` (§8). Thứ quyết định thành bại là **learning rate**, không phải họ optimizer (§11).

Đổi optimizer lúc fine-tune — nay đã có câu trả lời

Optimizer frontier có chạm tới người fine-tune không? Năm 2026 có hai bài báo trả lời thẳng — và chúng nói ngược nhau, vì **trả lời hai câu hỏi khác nhau**.

Pre-training (NVIDIA, 07/2026)

"SOAP và Muon liên tục vượt AdamW ở các quy mô chúng tôi thử" — model nhiều tỷ tham số, hàng nghìn tỷ token, batch tới **100M token**, nơi **AdamW suy giảm**.

arXiv 2607.20548. Phần tóm tắt **không** tuyên bố gì về fine-tuning.

Fine-tuning (05/2026)

*"Chuyển sang Muon để fine-tune một cách ngây thơ dẫn tới hiệu năng suy giảm do **lệch optimizer**"* — lệch này phá kiến thức đã pre-train và **tỷ lệ với độ mạnh của bước cập nhật**.

arXiv 2605.10468, "Can Muon Fine-tune Adam-Pretrained Models?"

Và đây là chỗ nó nối vào phần còn lại của buổi học

Khuyến nghị của chính bài báo đó: **dùng LoRA** khi buộc phải fine-tune bằng optimizer khác — *"LoRA thu hẹp khoảng cách giữa Adam và Muon quan sát được ở full fine-tune"*. Ràng buộc độ lớn cập nhật **chính là thứ bất thụ độ lệch**. Một lý do nữa để LoRA là mặc định nó thay thế cho những thứ full FT không thay

Kiến trúc 2026 — thưa hơn, và attention không còn là một thứ

VINUNIVERSITY

Độ thưa đã thành cực đoan

Model	% tham số kích hoạt
DeepSeek-V4-Pro	≈3,1%
Kimi K3	≈3,7%
Qwen3.8-2.4T	≈4,0%
GLM-5	≈5,4%

Kimi K3: 896 expert, chọn 16 + 2 dùng chung, 93 lớp. DeepSeek-V4 thêm **mHC** — ràng buộc residual lên một đa tạp để khôi phục ánh xạ đồng nhất.

Bốn họ attention cùng tồn tại

Họ	Ví dụ 2026
Softmax thưa/nén	DeepSeek **CSA** + **HCA**, **DSA**
Linear lai	**Gated DeltaNet** (Qwen3-Next, 3:1); **KDA** (Kimi K3)
Local/global xen kẽ	Gemma 4 (4:1 và 5:1)
GQA + RoPE thuần	model dense nhỏ — *thứ bạn thật sự fine-tune*

Mục tiêu chung: giữ trạng thái **kích thước cố định** để chi phí giải mã mỗi token không tăng theo độ dài ngữ cảnh.

Fine-tune model MoE — "ở đâu" giờ gồm cả "expert nào"

Định tuyến expert **lệch nặng**: một nhóm nhỏ expert nhận phần lớn token mỗi lớp. LoRA mặc định lại gắn adapter vào *mọi* expert — phần lớn adapter nằm chờ ở expert gần như không được gọi.

Định tuyến trước, rồi mới gắn adapter

Đếm số lần định tuyến trên một tập hiệu chuẩn nhỏ, chỉ gắn LoRA vào **top 25% expert được gọi nhiều nhất mỗi lớp**:

- chênh lệch chất lượng **trong ±1 điểm %**
- tham số huấn luyện **−70…73%**
- checkpoint adapter **−71…73%**
- thời gian huấn luyện **tới −50%**

Lưu ý: **Tín hiệu định tuyến mới là thứ tạo ra kết quả**, không phải ngân sách tham số: chọn expert **ngẫu nhiên** với cùng số tham số cho kết quả **kém hơn ≈2,5 điểm %**.

Hai lưu ý vận hành:

- **Đừng huấn luyện lớp router** — nhà cung cấp tắt mặc định vì lý do ổn định.
- Cùng một phép toán, khác bộ nhớ: PEFT *hiện thực hoá* delta LoRA cho mọi expert trước phép nhân MoE; Unsloth đổi thứ tự phép toán để tránh việc đó.

Đây là mở rộng tự nhiên của kết quả ở §11.2 — *vị trí* gắn adapter quan trọng hơn *rank*. Trên MoE, "vị trí" nay bao gồm cả "expert nào".

Khối khuếch tán — model không còn sinh từ trái sang phải

DiffusionGemma (Google DeepMind, 10/06/2026, Apache 2.0, **26B MoE / 3,8B kích hoạt**) là model ngôn ngữ **block-diffusion** có trọng số mở đầu tiên đủ lớn để fine-tune thật.

Cách nó sinh chữ • prefill: attention **nhân quả** đọc prompt
• sinh: attention **hai chiều** khử nhiễu lặp một **"canvas" 256 token** — *song song*
• canvas sạch → commit KV cache → block kế Tới **4**×

nhanh hơn khi sinh — 700+ tok/s trên RTX 5090, 1000+ trên một H100.

Ngày đầu: vLLM, Transformers, SGLang, MLX.

Công thức SFT — ba bất biến của deck *còn*, một *đổi*

Bất biến	Số phận
Vị trí > rank (§11.2)	**còn** — LoRA $r = 16$ trên attn + dense-MLP
Đừng train router (§7.5)	**còn** — "router MoE giữ đóng băng"
Merge rồi kiểm (§23)	**còn** — vẫn là PEFT
Che loss ở câu trả lời (§17)	**ĐỔI**

Vì sao "che loss" không còn nghĩa như cũ

Loss là **CE trung bình trên token được giám sát trong khối được chọn**, cộng một **encoder AR loss** trên chuỗi sạch ($t \sim U(0{,}001;\ 1)$). Không có "đoạn trả lời" trái-sang-phải để che — giám sát do **khối** và **lịch nhiễu** định nghĩa. Kỹ năng NB1 vẫn đúng; *cách* kiểm thì khác.

Full Fine-tune — Giải phẫu bộ nhớ

Tại sao cập nhật toàn bộ trọng số lại đắt đến thế — và đắt ở chỗ nào

08

Cập nhật mọi tham số

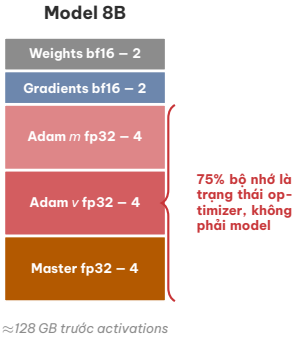
Khởi tạo từ trọng số pre-trained W_0, rồi chạy gradient descent trên **toàn bộ** tham số: mọi ma trận attention, mọi ma trận MLP, embedding, layer-norm — tất cả đều có gradient và đều được cập nhật.

Đây là "chuẩn vàng" về mặt biểu đạt: không có ràng buộc nào giới hạn hướng cập nhật. Cũng chính vì thế nó là phương án **tốn bộ nhớ nhất** — và bộ nhớ, chứ không phải tốc độ, mới là thứ chặn bạn lại.

Câu hỏi thật sự: một model 8B "chỉ" cần 16 GB để *chạy* ở bf16. Vậy tại sao fine-tune nó lại không vừa một card 80 GB?

Kế toán bộ nhớ — 16 byte cho mỗi tham số

Thành phần	Byte/param	Ghi chú
Trọng số (bf16)	2	bản đang tính
Gradient (bf16)	2	mỗi tham số một gradient
Adam *m* (fp32)	4	momentum bậc 1
Adam *v* (fp32)	4	momentum bậc 2
Master weights (fp32)	4	bản chính xác cao
Tổng	**≈16**	**chưa tính activations**



Quy tắc nhẩm — **≈16 GB trạng thái cho mỗi tỷ tham số**, cộng thêm activations.

Đây là lý do LoRA hiệu quả đến thế: adapter chỉ chiếm ~1% tham số, nên **ba hàng fp32 ở giữa co lại theo tỷ lệ đó** — phần đắt nhất biến mất.

Activations — thành phần bị quên và cách khống chế

Vì sao activations lớn:

- Backprop cần giá trị forward của **mọi** layer để tính gradient
- Tỷ lệ thuận với `batch` × `seq_len` × `hidden` × `layers`
- Không phụ thuộc full FT hay LoRA — **LoRA không cứu bạn khỏi khoản này**

Lưu ý: Sinh viên thường tính đủ chỗ cho trọng số rồi vẫn OOM. Thủ phạm gần như luôn là activations khi `seq_len` dài.

Gradient checkpointing

Không lưu activations trung gian — **tính lại** chúng trong lúc backward.

Đổi bộ nhớ lấy thời gian: giảm ∼60% activation memory, tăng ∼20% thời gian train.

Gần như luôn đáng bật khi bạn sát trần VRAM.

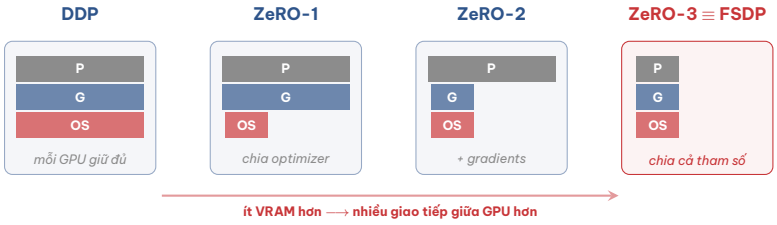
Ba đòn bẩy khác

Flash Attention: attention không vật chất hoá ma trận $n \times n$

Sequence packing: giảm token padding lãng phí (§17)

Activation offloading: đẩy sang CPU RAM

Chia nhỏ trạng thái — ZeRO & FSDP



P = parameters **G** = gradients **OS** = optimizer states

FSDP là bản gốc của PyTorch, tương đương ZeRO-3: mỗi rank giữ $1/N$ và *dựng lại* tensor đầy đủ khi cần dùng.

Offload

ZeRO-Offload / ZeRO-Infinity đẩy optimizer state (và cả tham số) sang CPU RAM hoặc NVMe — nâng sức chứa của một GPU lên nhiều lần, đổi lại băng thông PCIe trở thành nút thắt.

Khi nào Full FT vẫn là lựa chọn đúng

Chọn Full FT / CPT khi

- Dữ liệu quy mô **tỷ token** (CPT — §6)
- Dịch chuyển miền rất lớn: ngôn ngữ mới
- **Đổi tokenizer / mở rộng từ vựng**
- Bạn đang *tạo ra* một base model

Chọn PEFT khi

- Quy mô dữ liệu **post-training** thông thường
- Cần nhiều biến thể cho nhiều khách hàng
- Muốn giữ năng lực gốc (§6 — quên thảm hoạ)
- Ngân sách GPU thực tế

Điều đã thay đổi trong 2025–2026

Trước đây mặc định là "full FT tốt hơn, LoRA là bản rẻ tiền". Nghiên cứu 2025 cho thấy **trong vùng dữ liệu post-training, LoRA cấu hình đúng đạt *cùng* chất lượng** — phần lớn bằng chứng cũ đến từ *cấu hình sai* (§11).

Bậc thang giữa — full FT khi full FT không vừa

Giữa "full FT 16 byte/tham số" và "LoRA 0,3% tham số" có một bậc thang ít được dạy: giữ **mọi** trọng số huấn luyện được, nhưng nén **trạng thái optimizer**.

Cách	Cơ chế	Cái giá
Freeze-tuning	chỉ mở khoá *N* lớp trên cùng	đơn giản nhất; hạn chế năng lực
GaLore	chiếu *gradient* xuống hạng thấp, giữ moment trong không gian con	**SVD định kỳ**
BAdam	block coordinate descent — mỗi lúc chỉ một block hoạt động	nhiều bước hơn
APOLLO	thay SVD bằng **phép chiếu ngẫu nhiên**; APOLLO-Mini dùng scaling hạng 1	xấp xỉ
Adam-mini	**ít learning rate hơn** — một cho mỗi block thay vì mỗi tham số	moment bậc 1 vẫn đầy

Đây là *đối thủ* của LoRA, không phải bạn đồng hành

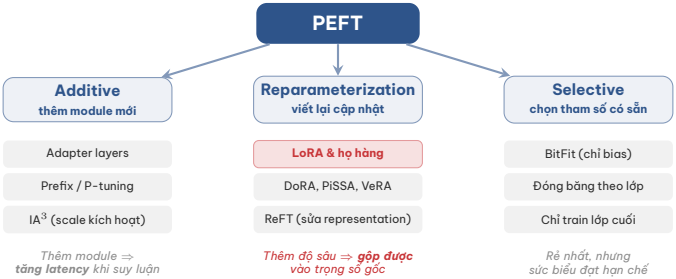
Dùng khi bạn **thật sự cần mọi trọng số dịch chuyển** — ví dụ CPT sang ngôn ngữ mới (§6). Với một tác vụ hẹp trên một GPU, LoRA vẫn thắng vì câu chuyện **merge & phục vụ** đơn giản hơn nhiều. Điểm yếu đã ghi nhận: **SVD đắt** (GaLore), đánh đổi chất lượng so với AdamW, và trạng thái optimizer **vẫn còn** (≈1/4 rank ở GaLore; moment bậc 1 đầy đủ ở Adam-mini).

09

Bản đồ họ PEFT

LoRA không phải phương án duy nhất — nhưng có lý do nó thắng

Ba nhánh của Parameter-Efficient Fine-Tuning



Vì sao LoRA thắng cuộc

Không phải vì học giỏi nhất, mà vì nó là nhánh duy nhất **gộp được vào trọng số gốc**: $W = W_0 + \frac{\alpha}{r}BA$. Đồ thị suy luận sau khi gộp *giống hệt* base model ⇒ **0% overhead latency**. Adapter layers thì luôn thêm một phép nhân ma trận vào mỗi forward pass.

“Thuật toán tối ưu” — một cái nhãn, bốn loại việc khác nhau

Danh sách nổi tiếng nhất trong hệ sinh thái (LLaMA-Factory) gộp 12 cái tên vào một dòng. Chúng **không cùng loại** — và phân loại lại chính là cách nhớ được chúng:

Loại	Đổi cái gì	Thành viên
A. Full FT tiết kiệm bộ nhớ	**optimizer**	GaLore · BAdam · APOLLO · Adam-mini · Muon
B. Hình dạng cập nhật	**tham số hoá** LoRA	DoRA · LoRA+ · OFT/OFTv2 · rsLoRA · GraLoRA
C. Khởi tạo	adapter **bắt đầu ở đâu**	PiSSA · LoftQ · OLoRA · EVA · CorDA
D. Phẫu thuật kiến trúc	**chính model**	LongLoRA · LLaMA Pro · Mixture-of-Depths

Lưu ý: **Loại D là cái bẫy.** LLaMA Pro *thêm lớp* (32→40), MoD cho token *bỏ qua* block, LongLoRA đổi mẫu attention lúc train — thứ chạy ra **không còn là kiến trúc base**. A, B, C **giữ nguyên** đồ thị.

Đặt câu hỏi này trước

Trước khi thử một cái tên mới, hỏi **nó ở loại nào**:

A → tôi có đang làm *full* FT không?

B/C → đổi một dòng config, an toàn

D → tôi có sẵn sàng nuôi một kiến trúc *phi tiêu chuẩn* không?

LoRA – Cơ chế hoạt động

Low-Rank Adaptation: giả thuyết “cập nhật hữu ích có hạng thấp”

LoRA (Hu et al., 2021)

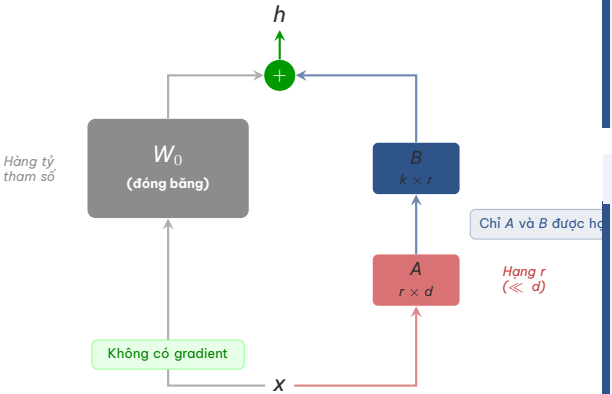
Đóng băng toàn bộ trọng số gốc W_0, chỉ học thêm một **cập nhật hạng thấp** $\Delta W = BA$:

$$h = W_0 x + \frac{\alpha}{r} BA\, x \qquad A \in \mathbb{R}^{r \times d},\ B \in \mathbb{R}^{k \times r},\ r \ll \min(d, k)$$

Giả thuyết nền tảng: cập nhật cần thiết để thích nghi một model đã pre-trained có *hạng nội tại thấp* — nó không cần đi khắp không gian $d \times k$ chiều, chỉ cần một không gian con nhỏ.

Ẩn dụ: như dán giấy nhớ lên sách giáo khoa — không sửa sách, chỉ ghi chú. Khi triển khai thì "in" ghi chú vào sách: $W = W_0 + \frac{\alpha}{r} BA$, và cuốn sách mới không dày hơn cuốn cũ.

Kiến trúc — Trọng số đóng băng + hai ma trận gầy



Khởi tạo — chi tiết quan trọng

$A \sim \mathcal{N}(0, \sigma^2)$ ngẫu nhiên, $B = 0$.

Nhờ $B = 0$ nên $\Delta W = 0$ tại bước đầu: model bắt đầu *đúng bằng* base model. Không có cú sốc chất lượng ở step 0.

Hệ số α/r

Tiền tố $\frac{\alpha}{r}$ giữ cho **độ lớn cập nhật trên mỗi thành phần hạng-1 không đổi** theo *r*.

Hệ quả (§11): learning rate tối ưu **gần như độc lập với rank**. Đổi *r* không bắt buộc phải dò lại LR.

Số tham số: $r(d + k)$ thay vì dk. Với $d = k = 4096$,

Bốn nút vặn, hai bậc tự do

Bạn tưởng mình có bốn siêu tham số độc lập:

$$\alpha, \quad \mathrm{LR}_A, \quad \mathrm{LR}_B, \quad \mathrm{init}_A$$

Nhưng động lực học **bất biến** dưới phép biến đổi:

$$\alpha \to \frac{\alpha}{pq}, \quad \mathrm{init}_A \to p \cdot \mathrm{init}_A,$$

$$\mathrm{LR}_A \to p \cdot \mathrm{LR}_A, \quad \mathrm{LR}_B \to q \cdot \mathrm{LR}_B$$

$\Rightarrow$ hai trong bốn nút là **dư thừa**. Không gian thật sự chỉ có **2 chiều**.

Lưu ý: Rất nhiều giờ GPU bị đốt để dò một lưới siêu tham số mà *một nửa số trục triệt tiêu lẫn nhau*. Hãy cố định α và init_A, chỉ dò learning rate và rank.

Quy ước thực dụng

Đặt $\alpha = 2r$ (tức $\alpha/r = 2$) rồi **không đụng đến nữa**.

Dò: `learning_rate` trước, `r` sau — và chỉ tăng r khi có bằng chứng thiếu năng lực (§11).

Nguồn: Thinking Machines Lab, *LoRA Without Regret* (2025)

LoRA Without Regret — Play-book 2026

Khi nào LoRA *bằng* full fine-tune — và ba lỗi cấu hình khiến nó không bằng

11

Thinking Machines Lab, 2025

LoRA đạt **cùng hiệu suất mẫu** và **cùng chất lượng cuối cùng** như full fine-tuning khi thoả *hai* điều kiện:

(1) LoRA được gắn lên **mọi ma trận trọng số**, đặc biệt là các lớp **MLP / MoE**;

(2) số tham số huấn luyện được **vượt lượng thông tin chứa trong tập dữ liệu**.

Ra khỏi vùng này, chất lượng suy giảm **từ từ** theo mức thiếu hụt năng lực — không có “vách đá”. Vùng không hối tiếc phủ *phần lớn* các tình huống post-training thực tế.

Lỗi #1 — Chỉ gắn LoRA vào attention

Thí nghiệm trên Llama-3.1-8B, số tham số *gần bằng nhau*:

Vị trí gắn	Rank	Params	Kết quả
Chỉ attention	256	0.25 B	kém hơn
Chỉ MLP	128	0.24 B	**tốt hơn**

MLP thắng **dù ít hơn 0.01 B tham số**. Đây không phải sai số — đây là hiệu ứng vị trí.

Vì sao: dưới góc nhìn empirical NTK, xấp xỉ

$\mathrm{eNTK}(\mathrm{LoRA}) \approx \mathrm{eNTK}(\mathrm{FullFT})$ chỉ đúng khi LoRA phủ các lớp *chứa phần lớn tham số*. Trong một LLM, đó là MLP — không phải attention.

Đừng làm

```
target_modules=["q_proj","v_proj"]
```

Cấu hình mặc định trong rất nhiều tutorial 2023–2024 — và là nguồn gốc của phần lớn lời đồn "LoRA yếu hơn full FT".

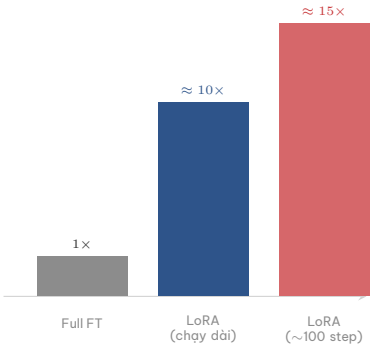
Hãy làm

```
target_modules="all-linear"
```

hoặc liệt kê đủ: `q,k,v,o_proj` + `gate,up,down_proj`

Nguồn: thinkingmachines.ai/blog/lora

Lỗi #2 — Dùng learning rate của full fine-tune



Learning rate tối ưu, tương đối so với full FT

Kết quả — LR tối ưu của LoRA **cao hơn full fine-tune khoảng 10 lần** — nhất quán ở cả supervised lẫn RL.

- Chạy ngắn (~100 step): nghiêng về **15×**, hội tụ về 10× khi train dài hơn
- Nhờ tiền tố α/r (§10), LR tối ưu **gần như độc lập với rank**

Lưu ý: Đây là lý do phổ biến nhất khiến một lần LoRA "không ăn thua": LR đặt theo thói quen full fine-tune, tức **thấp hơn tối ưu một bậc độ lớn**. Model học được rất ít, và ta đổ lỗi cho phương pháp.

Lỗi #3 — Tăng batch size để lấp đầy GPU

VINUNIVERSITY

LoRA nhạy với batch size hơn full FT

Khoảng cách chất lượng LoRA–FullFT **nới rộng khi batch size tăng**, và điều này **độc lập với rank**.

Đây là đặc tính của cách tham số hoá tích ma trận *BA* — **tăng *r* không sửa được**.

Ảnh hưởng thực tế có giới hạn (cả hai phương pháp đều đạt loss tốt nhất ở batch nhỏ), nhưng đây là cái bẫy cho ai quen "tăng batch cho đầy VRAM".

Chi phí tính toán

Full FT (forward + backward): $3N^2$ phép nhân–cộng

forward N^2 · backward $2N^2$

LoRA: $2N^2 + 6NR$

Với $R \ll N$: **nhỉnh hơn** $\frac{2}{3}$ **của** $3N^2$ **một chút**.

Lưu ý: Đừng nhầm hai con số: $\frac{2}{3}$ là **FLOPs**. Khoản tiết kiệm **bộ nhớ** lớn hơn nhiều (§8) vì trạng thái optimizer co theo số tham số *huấn luyện được*, không phải tổng tham số.

Với RL, rank 1 là đủ — và đây là cầu nối sang Ngày 22

Lập luận thông tin: thuật toán policy-gradient hấp thụ khoảng **1 bit thông tin mỗi episode**:

$$I(G; R \mid \text{history}) \leq H(\text{Adv}) \lesssim \log B$$

B = số bin của advantage $\Rightarrow O(1)$ bit / episode

Ví dụ trên MATH:

- $\sim$10.000 bài $\times$ 32 mẫu $=$ **320.000 bit** tổng thông tin
- LoRA **rank 1** trên Llama-3.1-8B $=$ 3M tham số
- Dư năng lực gần **10**$\times$ $\Rightarrow$ rank-1 **bằng** full FT

Lặp lại trên Qwen3-8B-base + DeepMath: đường học trùng nhau, cả hai đều phát triển hành vi suy luận nâng cao (quay lui, tự kiểm chứng).

Trực giác

SFT nhồi vào model *cả một tập dữ liệu* — cần nhiều năng lực.

RL chỉ truyền về *một tín hiệu thưởng rất mỏng* mỗi lần thử — cần rất ít năng lực.

Vì thế RL **gần như miễn phí** về mặt tham số huấn luyện.

Ngày 22 tiếp nối

Hôm nay: *cơ chế* — vì sao adapter nhỏ xíu vẫn đủ cho RL.

Ngày mai: *thuật toán* — DPO, ORPO, GRPO, RLVR.

Hoà giải hai kết quả tưởng như mâu thuẫn

Biderman et al., 2024

"LoRA Learns Less and Forgets Less"

LoRA **kém hơn** full FT trên tác vụ đích...
...nhưng **giữ lại nhiều năng lực gốc hơn**.

Chống quên tốt hơn cả weight decay lẫn dropout; giữ được độ đa dạng sinh văn bản.

Thinking Machines, 2025

"LoRA Without Regret"

LoRA **bằng** full FT — với điều kiện gắn đủ lớp (kể cả MLP) và LR chia đúng thang ($\sim 10\times$).

Cả hai đều đúng — và đây là bài học

Hai kết quả không mâu thuẫn: phần lớn bằng chứng "LoRA học kém hơn" đến từ các thiết lập **chỉ gắn attention** và dùng **learning rate cỡ full fine-tune** — đúng hai lỗi ở §11.2 và §11.3. Sửa cấu hình thì khoảng cách *học* biến mất; nhưng lợi thế **chống quên** thì vẫn còn thật, và đó vẫn là một lý do chính đáng để chọn LoRA.

Biến thể LoRA

12

DoRA, rsLoRA, PiSSA, OFT, GraLoRA — và bằng chứng đo được năm 2026

Bản đồ biến thể — mỗi biến thể sửa một chỗ

Biến thể	Sửa điều gì	Khi nào dùng
rsLoRA	Đổi thang $\alpha/r \to \alpha/\sqrt{r}$ (rank-stabilized)	Khi dùng rank cao; bật gần như miễn p
DoRA	Tách cập nhật thành *độ lớn* + *hướng*	Mặc định tốt; **cần PEFT $\geq$ 0.10** khi me
LoRA+	LR khác nhau cho A và B (tỷ lệ $\sim$2:1 nghiêng về B)	Hội tụ nhanh hơn; là ca riêng của bất b
PiSSA	Khởi tạo A, B từ *thành phần chính* của W thay vì ngẫu nhiên	Khi thời gian huấn luyện là ràng buộc
VeRA	Dùng chung cơ sở ngẫu nhiên đóng băng + vector tỷ lệ rất nhỏ	Khi cần cực kỳ tiết kiệm tham số (nhiề

Lưu ý: **DoRA + merge:** vector độ lớn phải được gộp lại đúng cách khi merge adapter. Với PEFT cũ hơn 0.10, thành phần này bị áp dụng sai — model đã merge *không* giống model lúc train.

Cấu hình khởi điểm 2026

```
r=16, target_modules="all-linear"
```

tuỳ chọn thêm rsLoRA hoặc DoRA. Đo trước, đổi sau.

Sự thật khó chịu về "vườn thú biến thể"

Đổi sang DoRA/PiSSA *không* đảm bảo cải thiện lớn

Các so sánh độc lập cho kết quả **lẫn lộn**: PiSSA cải thiện với model này, kém đi với model khác. Nhiều ứng dụng cho thấy mọi biến thể hội tụ về gần cùng một điểm.

Mỗi năm có hàng chục biến thể mới với bảng benchmark đẹp. Rất ít sống sót qua kiểm chứng độc lập trên tác vụ khác.

Thứ tự ưu tiên thật sự

1. Chất lượng dữ liệu (§17)
2. Che loss đúng chỗ — template & masking (§17)
3. Phủ đủ lớp: `all-linear` (§11.2)
4. Learning rate đúng thang (§11.3)
5. Rank
6. ...rồi mới đến chọn biến thể

Bài học

Bốn mục đầu quyết định phần lớn kết quả và **đều miễn phí**. Nếu bạn đang cân nhắc DoRA *trước khi* kiểm tra mask loss, bạn đang tối ưu sai chỗ.

Bằng chứng 2026 — và hai bài báo tưởng như đối nghịch

A. Đội PEFT của HuggingFace (06/2026): *"LoRA không nên là mặc định tự động"*. Llama-3.2-3B, MetaMathQA → GSM8K:

Phương pháp	GSM8K	VRAM
Lily	**54,9%**	25,6 GB
LoRA (khởi tạo **rsLoRA**)	53,2%	22,6 GB
LoRA (chuẩn)	48,1%	22,5 GB
LoRA-FA	48,1%	20,2 GB
BEFT	32,9%	20,2 GB

Sinh ảnh (FLUX.2-klein-4B): **OFT 0,708 @ 9,01 GB** so với LoRA 0,697 @ 9,97 GB — *áp đảo trên cả hai trục.*

B. "Learning Rate Matters" (arXiv 2602.04998): quét siêu tham số cho **9 biến thể** + LoRA gốc trên toán, code, commonsense, instruction.

> *"Một khi learning rate được tinh chỉnh đúng, mọi phương pháp đạt đỉnh **trong khoảng 1–2%** của nhau"* — và **mỗi biến thể ưa một dải LR khác nhau**.

Các cải thiện đã công bố *"có thể không phản ánh ưu thế phương pháp"* — vì so sánh dùng siêu tham số cố định.

Hoà giải — và đây mới là bài học

Nhìn lại bảng A: **LoRA chuẩn 48,1% → 53,2% chỉ vì đổi cách chia thang** sang rsLoRA ($\alpha/\sqrt{r}$ thay cho α/r) — tức là đổi **learning rate hiệu dụng**. Chênh lệch lớn nhất trong "bảng vượt LoRA" **+5,1 điểm** là hiệu ứng LR; phương pháp lạ nhất (Lily) chỉ hơn baseline đã sửa **+1,7**. *Hai bài báo không mâu thuẫn — chúng xếp*

Hai thứ *thật sự* khác — và một cơ chế giải thích trần rank

VINUNIVERSITY

OFT / BOFT — cập nhật *nhân*, không phải *cộng*

LoRA: $W + BA$. OFT: $R \cdot W$ với R **trực giao** — giữ nguyên **hyperspherical energy** của trọng số gốc, tức chống quên bằng *cấu trúc*, không phải regularizer.

- Merge bình thường (`merge_and_unload`)
- `oft_block_size=32`, `use_cayley_neumann`
- Nhiều tham số hơn LoRA

GraLoRA — vì sao tăng rank hết tác dụng

Một cổ chai r duy nhất gây **rối gradient** giữa các kênh không liên quan — lý do chất lượng **chững hoặc tụt trên rank 32–64**.

GraLoRA chia ma trận thành **khối con**, mỗi khối một adapter — chi phí "không đáng kể", tới **+8,5 Pass@1** trên HumanEval+.

Thứ tự thử — đã cập nhật

1. Sửa thang LR (và bật `rsLoRA`) — *miễn phí, hiệu ứng lớn nhất đo được* **2.** Khởi tạo tốt hơn: PiSSA / LoftQ (4-bit) / EVA **3.** *Rồi mới* đổi cơ chế: OFT (chống quên), GraLoRA (rank cao).

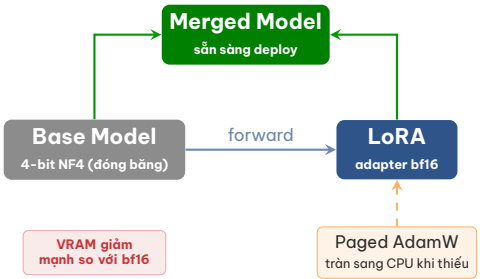
PEFT nay có hơn **20** phương pháp họ LoRA — học **các trục** thì đặt được bất kỳ tên mới nào vào đúng chỗ.

QLoRA & Lượng tử hoá khi huấn luyện

4-bit cho base đóng băng — và biên giới mới: huấn luyện thẳng ở FP8/FP4

13

QLoRA — NF4 4-bit + adapter bf16



QLoRA (Dettmers et al., 2023) — Lượng tử hoá base xuống 4-bit NF4, gắn adapter bf16 lên trên. Chỉ adapter được huấn luyện.

- **NF4**: kiểu dữ liệu 4-bit tối ưu cho trọng số phân phối chuẩn — không phải int4 thông thường
- **Double quantization**: lượng tử hoá cả *hằng số lượng tử hoá* — tiết kiệm thêm
- **Paged optimizer**: đẩy trạng thái optimizer sang CPU RAM khi VRAM chạm đỉnh, thay vì OOM

Lưu ý: Gradient vẫn chảy *qua* base đã lượng tử hoá để tới adapter — base 4-bit

Thang chi phí — Full FT vs LoRA vs QLoRA (2026)

Phương pháp	Model ∼8B	Model ∼70B	Params huấn luyện
QLoRA (4-bit)	≥12 GB *(RTX 3060)*	∼48 GB *(1 GPU)*	∼1%
LoRA (bf16)	∼28 GB	>140 GB *(riêng trọng số)*	∼1%
Full FT (bf16+AdamW)	∼130 GB ⇒ đa GPU	∼1.120 GB	100%

Điều đáng nhớ là tỷ lệ

Full FT tốn **hơn 10**× QLoRA về bộ nhớ (16 byte/tham số, §8, chưa gồm activation). Con số GB tuyệt đối thay đổi theo từng thế hệ phần cứng và từng phiên bản thư viện — **hãy tự đo lại**, đừng chép bảng này vào báo cáo.

Lưu ý: **Rẻ nhất không còn tự động là đúng nhất.** Với vài model mới nhất (vd. dòng Qwen3.5), nhà cung cấp *khuyến nghị không dùng* QLoRA — sai số lượng tử hoá cao hơn bình thường — và đề xuất **LoRA bf16**, nhất là với biến thể MoE.

FSDP + QLoRA không loại trừ nhau: shard *và* lượng tử hoá cùng lúc — cách đưa 70B QLoRA về tầm một máy vài GPU.

RTX 5090 32GB: LoRA đến ∼13B fp16, hoặc QLoRA ∼32B. B200: 180 GB HBM3e. Nguồn: tổng hợp khảo sát phần cứng 2026 — xem RESEARCH

Biên giới — huấn luyện thẳng ở FP8 và 4-bit

4-bit đang chuyển từ "mẹo nén để lưu trữ" thành **kiểu dữ liệu tính toán** cho cả huấn luyện lẫn suy luận.

Định dạng	Đặc điểm
MXFP4	khối 32 phần tử, một mức tỷ lệ
NVFP4	khối **16**, tỷ lệ **hai mức**: E4M3 theo khối + FP32 theo tensor ⇒ sai số thấp hơn

So với FP8: NVFP4 cho **2–3**× thông lượng số học, ∼**1.8**× tiết kiệm bộ nhớ; pretrain Llama-3.1-405B nhanh hơn ∼**1.9**×. **2026 — QAT đã nằm trong trọng số bạn tải về.**

Kimi K3 huấn luyện **MXFP4 cho trọng số / MXFP8 cho activation** *từ giai đoạn SFT trở đi*; NVIDIA báo cáo pretrain ổn định ở **NVFP4** tới **25 T token**.

Lưu ý: **Không phải phép toán nào cũng hạ được độ chính xác.** Optimizer, normalization và softmax phải ở lại độ chính xác cao — nếu không, huấn luyện mất ổn định và không hội tụ.

Phân biệt ba khái niệm

PTQ — lượng tử hoá *sau* khi train, không cần huấn luyện lại

QAT — mô phỏng lượng tử hoá *trong lúc* train để model quen với sai số

QLoRA — base lượng tử hoá + adapter độ chính xác cao; không phải QAT

Ma trận QLoRA đa-bit — *nạp-và-train*, không phải *train-và-giao*

VINUNIVERSITY

Backend	Bit (train)	Hiệu chuẩn	Merge lại?	Ghi chú của tài liệu
bitsandbytes (=*LLM.int8*)	8, **4 (NF4)**	không	**có**	đường QLoRA chuẩn của PEFT
GPTQ (GPTQModel)	2, 3, 4, 8	**có** (≈20 ph/8B)	có	nhiều checkpoint dựng sẵn
HQQ	8/4/3/2/1	không	có	dưới 4-bit tụt rõ
EETQ	8 (int8)	không	có	tuyên bố nhanh hơn LLM.int8
AQLM	≈2	có (rất đắt)	**KHÔNG**	"không thể merge với trọng số AQLM"
torchao	**chỉ int8**	không	chỉ `int8_weight_only`	`int4` chưa hỗ trợ; NF4 chưa có
INC (Gaudi)	FP8	có	**KHÔNG**	`merge()` không hỗ trợ
Transformer Engine	FP8	—	**KHÔNG**	DoRA cũng không

Lưu ý: **Ba trong tám dòng không merge được**, dòng thứ tư chỉ merge đúng ở một dtype — ma trận liệt kê thứ bạn *nạp và train* được, không phải thứ bạn **giao đi** được (chính là cổng NB6 của lab). Thêm hai chi tiết: **AWQ không có** trong danh sách train của PEFT — nó là định dạng suy luận; và **không backend nào train ở 5 hay 6 bit** — chuỗi "2/3/4/5/6/8" là *hợp* của nhiều backend, 5/6-bit đến từ k-quant GGUF.

Siêu tham số & Vòng lặp huấn luyện

Từ cấu hình đến đường loss biết nói

Toàn cảnh pipeline



Vòng lặp, không phải đường thẳng

Mũi tên đứt nét mới là phần tốn thời gian thật. Hãy thiết kế để **quay lại bước 2 nhanh**: checkpoint thường xuyên, ghi lại cấu hình, và giữ tập eval cố định để hai lần chạy so sánh được với nhau.

Bảng siêu tham số khởi điểm

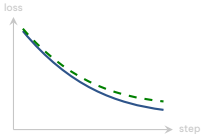
Tham số	Khởi điểm	Lý do / khi nào đổi
`target_modules`	`"all-linear"`	Phủ MLP là điều kiện của vùng không hối tiếc (§11.2)
`r`	16 | **256**	16 cho tác vụ hẹp; **256** khi dữ liệu ở quy mô post-train
`lora_alpha`	$2r$	Cố định rồi thôi — xem bất biến §10.3
`learning_rate`	$\approx 10\times$ LR full FT	Thang đúng quan trọng hơn giá trị tuyệt đối (§11.3)
LR schedule	cosine + warmup 5–10%	Warmup tránh sốc gradient ở bước đầu
`num_epochs`	1–3	>3 gần như luôn là quá khớp trên tập nhỏ
Batch hiệu dụng	$< \mathbf{32}$	LoRA kém dung nạp batch lớn — tăng rank *không* bù đ
`packing`	**tắt** nếu nhãn đã token hoá sẵn	Packing nối mẫu $\rightarrow$ phá căn chỉnh nhãn (§17.3)
`padding_free`	**chỉ khi** có FlashAttention *và* batch ≥ 2	FA-2 đòi **sm_80+**; T4 không có (§15.1)
`loss_type`	`"chunked_nll"`	Mặc định SFT của TRL từ v1.7 — giảm $\sim$30–50% VRAM

Lưu ý: Bảng này là **điểm xuất phát**, không phải đáp án — không gì thay được việc đo trên eval của chính bạn.

Đã thành tài liệu chính thức

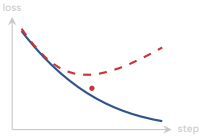
Khuyến nghị ở §11 nay nằm trong **tài liệu chính thức của TRL** (`lora_without_regret`), kèm script tái lập.

Đọc đường loss — ba hình dạng cần nhận ra



Lành mạnh

Cả hai cùng giảm rồi phẳng ra



Quá khớp

Train giảm, **eval quay đầu** ⇒ dừng ở điểm đỏ



Gai loss

LR quá cao, dữ liệu bẩn, hoặc tràn số ở bf16

Phản ứng đúng với từng hình

Quá khớp → giảm epoch, tăng dữ liệu, hoặc giảm rank — *không* phải giảm learning rate theo phản xạ. **Gai loss** → bật gradient clipping, kiểm tra vài mẫu quanh bước đó, xem lại LR. **Loss phẳng ngay từ đầu** → gần như luôn là **mask sai** (§17.2) hoặc LR thấp hơn tối ưu một bậc (§11.3).

Tăng tốc — Attention, Kernel, Loss & Backward

Cùng một phép toán, khác cách thực thi — và vì sao GPU của bạn quyết định điều đó

Kernel *không phải* kiến trúc — hai trục khác nhau

§7.4 nói **model là gì** (CSA/HCA, Gated DeltaNet, KDA, local:global). Mục này nói **phép toán đó được chạy ra sao**. FlashAttention **không đổi** kết quả attention — nó đổi lượng dữ liệu phải đi qua HBM. Nhầm hai trục này là lỗi phổ biến khi đọc changelog.

Thang thế hệ FlashAttention — là hợp đồng phần cứng

Thế hệ	GPU tối thiểu	Ghi chú
FA-2	**Ampere sm_80+**	A100, 3090, 4090, L4
FA-3	Hopper (H100)	warp specialization, FP8
FA-4	Blackwell (B200)	CuTe-DSL, 03/2026
không có	**Turing sm_75**	**T4 — Colab Free**

FA-4 (arXiv 2603.05451, Tri Dao và cộng sự): tới **1613 TFLOPs/s** = 71% hiệu suất phần cứng trên B200; 1,3× so với cuDNN 9.13. Viết *hoàn toàn* bằng CuTe-DSL nhúng trong Python.

Lưu ý: `padding_free` cần kernel FlashAttention. Trên **T4** nó **không bao giờ** bật được — không phải vì cấu hình sai, mà vì sm_75 không có FA. Đây là lý do Lab 21 tắt cờ này thay vì bật một cờ vô tác dụng.

Quy tắc đọc

Toán tử tăng tốc = hợp đồng với phần cứng, **không phải** cờ cấu hình. Trước khi bật, hỏi: *GPU của tôi có compute capabil-*

Loss — lớp tốn bộ nhớ nhất không phải attention

Với từ vựng **150K–260K token**, tensor logits `[batch × seq × vocab]` là activation **lớn nhất** trong đồ thị — lớn hơn cả hidden state. Đây là chỗ "loss" thôi là công thức toán và trở thành hoá đơn VRAM.

Cut Cross-Entropy (Apple, ICLR 2025)

Chỉ tính logit của **token đúng**, còn log-sum-exp trên toàn từ vựng thì tính *on-the-fly* trong bộ nhớ nhanh — logits đầy đủ **không bao giờ tồn tại**.

Đo trên **Gemma 2**:

- bộ nhớ tính loss: **24 GB → 1 MB**
- classifier head: **28 GB → 1 GB**

Cùng một ý tưởng, ba cái tên bạn sẽ gặp

Tên	Ở đâu
`chunked_nll`	TRL — **bạn đã bật ở §14**
FLCE	Liger Kernel (LinkedIn)
CCE	`apple/ml-cross-entropy`

Liger hợp nhất **cả forward lẫn backward** của CE vào một kernel Triton: giảm $>4\times$ đỉnh bộ nhớ ở vocab 128K.

Vì sao điều này quan trọng với bạn

`loss_type="chunked_nll"` ở §14 — **đây là cơ chế đằng sau cờ đó**. Không phải mẹo vặt: nó là lý do một model 4B fine-tune vừa 16 GB.

Backward — bốn cần gạt, xếp theo cái giá phải trả

Cần gạt	Cơ chế	Giá
Loss hợp nhất	không bao giờ hiện thực hoá logits	**miễn phí**
Gradient checkpointing	lưu activation "mỏ neo", tính lại phần còn lại	$\approx +30\%$ thời gian
Checkpointing *chọn lọc*	giữ op đắt (attention, collective), tính lại op rẻ	đánh đổi tinh chỉnh
Activation offload	đẩy activation sang RAM CPU giữa fwd và bwd	băng thông PCIe
Gradient accumulation	đổi wall-clock lấy batch hiệu dụng	*nhưng* trần batch §11.4

Lưu ý: **Đóng băng base *không* bỏ được backward.** Gradient vẫn phải chảy *xuyên qua* lớp đóng băng để tới adapter nằm sâu hơn. Thứ biến mất là **trạng thái optimizer và bộ nhớ gradient** của tham số đóng băng — không phải phép tính backward.

Hệ quả thực tế

LoRA tiết kiệm **bộ nhớ**, không tiết kiệm **FLOPs của backward**. Ai kỳ vọng "LoRA nhanh gấp nhiều lần" vì chỉ train 0,3% tham số sẽ thất vọng khi đo: bước lùi vẫn đi hết chiều sâu model.

Bốn "toán tử tăng tốc" — và con số nào kiểm chứng được

Công cụ	Thật sự làm gì	Trạng thái kiểm chứng
FlashAttention-2/3/4	giảm lưu lượng HBM của attention; toán học không đổi	**kiểm chứng được** — có bài báo, có kernel, gắn với sm_
Liger Kernel	kernel Triton hợp nhất (RMSNorm, RoPE, SwiGLU, **FLCE**)	công bố: +20% thông lượng, −60% bộ nhớ
Unsloth	**viết tay đạo hàm** thay autograd; đổi thứ tự phép toán MoE	**tuyên bố của nhà cung cấp** — "2× nhanh, −70% VRAM" chưa được tái lập độc lập
KTransformers	đẩy gradient của lớp đóng băng sang **CPU**, expert scheduling	đổi *cái gì vừa*, không phải *nhanh bao nhiêu*

Lưu ý: Lợi thế **một GPU** của Unsloth **không** chuyển sang đa GPU — chính trang PyPI của nó ghi multi-GPU "đang cải thiện". Cơ chế thì thật; con số headline thì chưa ai ngoài nhà cung cấp đo lại.

Cách đọc một tuyên bố tăng tốc

1. Nó đổi *toán học* hay chỉ đổi *cách chạy*?
2. Đo trên phần cứng nào, batch nào, seq nào?
3. Có ai *ngoài* tác giả tái lập chưa?

Hệ sinh thái công cụ & Theo dõi thí nghiệm

Ma trận tính năng là thực đơn do nhà bếp viết — đọc nó như thế nào

16

Bốn khung fine-tuning — chọn theo *ràng buộc* của bạn

Khung	Hình dạng	Chọn khi
TRL (HuggingFace)	thư viện Python, `SFTTrainer`	bạn muốn **thấy** vòng lặp huấn luyện — *Lab 21 dùng cái này*
LLaMA-Factory	YAML + CLI + GUI (**LlamaBoard**), 100+ model	cần quét nhiều cấu hình / nhiều model nhanh
Unsloth	kernel viết tay, một GPU	bị bó VRAM trên **một** GPU tiêu dùng
NeMo / TorchTitan / Megatron	nhiều node, FSDP2 + expert parallel	huấn luyện quy mô cụm — và là đường chính thức cho MoE lớn

Vì sao lab dùng TRL trần

Một khung YAML giấu đi đúng những thứ ngày hôm nay dạy: mask loss, template, thang LR, vị trí adapter. Bạn **sẽ** dùng LLaMA-Factory trong công việc — nhưng chỉ sau khi biết nó đang giấu cái gì.

Lưu ý: **KTransformers** là backend cắm vào LLaMA-Factory (cùng config), đẩy gradient của lớp đóng băng sang CPU — nó đổi **cái gì vừa VRAM**, không phải cái gì nhanh nhất. Xem §15.4.

Đọc ma trận tính năng — ba câu hỏi nó không trả lời

Trích nguyên văn README của LLaMA-Factory (08/2026) — và ba chỗ nó *im lặng*:

Mục	README nói	Ma trận im lặng về
Độ chính xác	16-bit full/freeze/LoRA + **2/3/4/5/6/8-bit** QLoRA qua AQLM/AWQ/GPTQ/LLM.int8/HQQ/EETQ	**merge được hay không** (§13)
Thuật toán	GaLore, BAdam, APOLLO, Adam-mini, Muon, OFT, DoRA, LongLoRA, LLaMA Pro, MoD, LoRA+, LoftQ, PiSSA	chúng là **bốn loại việc khác nhau** (§9)
Tăng tốc	FlashAttention-2, Unsloth, Liger Kernel, KTransformers, RoPE scaling, NEFTune, rsLoRA	**GPU tối thiểu** của từng cái (§15.1)
Engine	vLLM worker *hoặc* SGLang worker	—
Theo dõi	LlamaBoard, TensorBoard, Wandb, MLflow, SwanLab	—

Lưu ý: **Có mặt trong ma trận ≠ được dùng.** LongLoRA, LLaMA Pro, MoD vẫn được liệt kê — nhưng **không** tech report frontier nào trong tập nguồn của deck dùng chúng. Danh sách chứng minh *bảo trì*, không phải *áp dụng*.

Nó cũng *thay đổi*

So với danh sách phổ biến 2024, README nay thêm **APOLLO, Adam-mini, Muon, OFT**, **Liger Kernel, KTransformers** và **SGLang**. Tài liệu nào thiếu chúng là đã cũ.

Theo dõi thí nghiệm — giá trị nằm ở *cấu hình*

Bức tranh 2026

Công cụ	Vị trí
W&B	dashboard nhóm, cloud-first
MLflow	nguồn mở, self-host, registry
TensorBoard	debug cục bộ, thời gian thực
SwanLab	nguồn mở, giám sát cả phần cứng
LlamaBoard	*không cùng hạng* — GUI của LLaMA-Factory

Nhiều đội dùng **đồng thời**: TensorBoard để debug, MLflow làm registry, W&B cho cả nhóm nhìn.

Vì sao mục này nằm trong một buổi fine-tuning

Mọi lỗi ở §22 đều là run có **đường loss đẹp**. Giá trị của tracker không phải biểu đồ — mà là **cấu hình được ghi cạnh kết quả**: mask mode, LR, target modules, ngân sách step, seed.

Thí nghiệm bạn không dựng lại được thì không phải thí nghiệm — nó là một giai thoại có biểu đồ.

Lab 21 ép bạn làm bằng tay

`results/runs.csv` + cổng `max_steps` dùng chung. Bốn run chỉ so sánh được khi **cùng số step** — và

Thuê người khác fine-tune — bốn *kiểu* dịch vụ

Kiểu	Ví dụ (10/2026)	Bạn điều khiển	Tính tiền theo
Job công thức sẵn	Vertex (Gemini SFT; RFT preview — *chỉ thử nghiệm, không production*), Bedrock (Nova 2 Lite; RFT cho gpt-oss, Qwen3), Azure Foundry, Together	dữ liệu + vài siêu tham số	token train, hoặc giờ (RFT)
API huấn luyện lập trình được	Tinker, Fireworks Training API (GA 08/2026)	vòng lặp, loss, sampling, rank	token prefill / sample / train
GPU chạy code của bạn	HF Jobs, Modal, SageMaker training job, Anyscale	mọi thứ	giờ GPU
Đang đóng cửa	OpenAI self-serve (§1), Mistral hosted FT (deprecated), HF AutoTrain (“no longer maintained”)	—	—

Một con số thật: Gemini 3.5 Flash SFT/RFT **$10 / 1M token train**; 3.1 Flash-Lite SFT **$3**. Bedrock RFT trên model mở dùng **LoRA rank 32**.

Lưu ý: “Có trong catalog” ≠ “tune được ở đó”. Và “managed” có thể chỉ là *thuê GPU* — hỏi ai viết vòng lặp huấn luyện.

Phiên bản 10/2026 — và vì sao Lab 21 *không* chạy theo

Gói	Lab 21 (08/2026)	Mới nhất	Ngày
transformers	5.15.1	**5.18.0**	30/09
TRL	1.10.0	**1.14.1**	29/09
PEFT	0.20.0	**0.21.2**	01/10
Axolotl	—	0.20.0	30/09
LLaMA-Factory	—	0.9.5	30/05
torchtune	—	0.6.1	04/2025
vLLM	—	0.31.0	05/10
SGLang	—	0.5.21	02/10

Đáng biết: transformers 5.18 thêm **FSDP2 + expert parallel** (mesh 2 chiều) — đường chính thức cho MoE; PEFT 0.21.2 đòi transformers ≥ 5.18; Axolotl 0.20 bỏ FSDP1.

torchtune không có bản mới từ 04/2025 — chưa có thông báo ngừng, nhưng đừng chọn nó cho dự án mới.

Lab pin bản cũ là có chủ đích

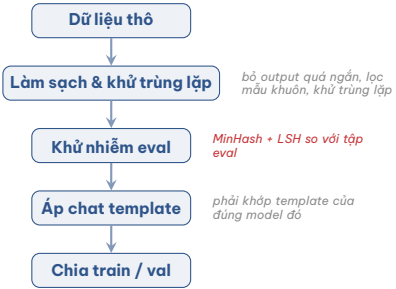
> Pin cả bộ đã chạy thật trên T4. Nâng một gói (PEFT 0.21) kéo theo gói khác (transformers 5.18) — đó là cách một lab đang chạy tốt hỏng giữa buổi.

Dataset — Chất lượng, Template & Masking

17

Nơi phần lớn các lần fine-tune thất bại — và không ai nhận ra

Quy mô dữ liệu — chất lượng đè bẹp số lượng



Cần bao nhiêu mẫu?

- Phong cách / định dạng: **500–2.000** mẫu chất lượng cao
- Thích nghi miền: **10.000+** mẫu
- Quy mô continued-pretraining: **tỷ token** — lúc đó xem lại §8 và §6

> **Quy tắc — 500 mẫu hoàn hảo > 10.000 mẫu nhiễu.** Một mẫu sai dạy model một thói quen sai — và nó học rất nhanh.

Vì sao ít mà tinh lại thắng: model đã biết ngôn ngữ. Bạn không dạy nó tiếng Việt — bạn chỉ cho nó thấy *dạng câu trả lời bạn muốn*. Vài trăm ví dụ nhất quán làm việc đó tốt hơn hàng vạn ví dụ mâu thuẫn nhau.

Che loss — nút vặn quan trọng nhất mà ít ai dạy

```python
from trl import SFTTrainer, SFTConfig

cfg = SFTConfig(
    max_length=2048,           # KHONG phai max_seq_length
    packing=True,
    packing_strategy="bfd",    # best-fit-decreasing
    padding_free=True,         # can FlashAttention (S15)
    completion_only_loss=True, # prompt/completion data
    # assistant_only_loss=True # multi-turn: needs {%
          generation %}
)
trainer = SFTTrainer(
    model=model, args=cfg,
    train_dataset=ds,
)
trainer.train()
```

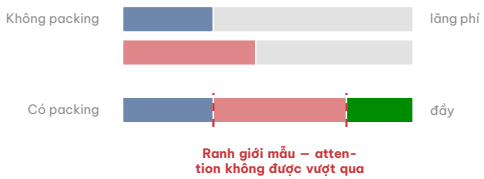
Lưu ý: Loss trên *cả* prompt = dạy model **sinh câu hỏi của bạn**. Muốn học câu trả lời: chỉ tính loss trên câu trả lời.

Nghi thức kiểm tra mask

Giải mã ngược các vị trí `mask == 1`; phải *đúng bằng* nội dung assistant. 5 phút, cứu một run 8 tiếng.

`assistant_only_loss` cần thẻ `{% generation %}`; Qwen3.5 gốc thiếu ⇒ mask **rỗng** (Lab F-10). TRL ≥1.10 tự vá hoặc **báo lỗi**; mask *tự dựng* thì im lặng.

Sequence packing — tăng tốc miễn phí, *có điều kiện*



Packing — Ghép nhiều mẫu ngắn vào một chuỗi dài để không lãng phí token padding. Thông lượng tăng đáng kể.

Lưu ý: **Nhiễm chéo:** nếu attention được phép nhìn xuyên ranh giới, mẫu B "đọc trộm" mẫu A. Model học được mối liên hệ không tồn tại, và bạn sẽ không thấy điều đó trong loss.

Cách sửa: `padding_free=True` + kernel attention hiểu độ dài thay đổi (FlashAttention varlen), để position id được reset ở mỗi ranh giới.

Nhiễm benchmark

Benchmark lọt vào tập train **thổi phồng điểm**.

Giả thuyết (chưa đo trực tiếp): teacher từng thấy benchmark ⇒ dữ liệu nó sinh cho bạn có thể mang theo nhiễm.

Giảm: MinHash + LSH *phát hiện* gần-trùng (không bảo đảm sạch); eval riêng trên dữ liệu thật; ghi provenance teacher.

Trôi phân phối ≠ sụp đổ model

Trôi phân phối phổ biến hơn collapse: dữ liệu tổng hợp sạch sẽ, người dùng thật gõ sai chính tả. Triệu chứng: benchmark đẹp, production tệ. Collapse: §18.5.

Nguyên tắc

Tổng hợp để *mở rộng* phần đuôi hiếm, không *thay thế* dữ liệu thật; eval luôn gồm **đầu vào thật** (§18).

Fine-tune model biết suy luận — cái bẫy không báo lỗi

Model mở 2026 đều có chế độ suy luận (§2). Fine-tune bằng dữ liệu *hỏi → đáp* thông thường sẽ **phá huỷ suy luận trong khi accuracy vẫn tăng** — không chỉ số quen thuộc nào báo động.

Model / tác vụ	Cách biểu diễn	Trace hợp lệ
Qwen3-8B, hoá học	khối rỗng	**0%**
Llama-R1-8B, GSM8K	bỏ hẳn khối	58,2%
Llama-R1-8B, GSM8K	khối rỗng	**100%**
Olmo-3-7B	cả hai	trace *bị cụt*

Qwen3-8B vẫn đạt 56,6% accuracy — vừa học việc mới, vừa mất sạch suy luận.

Chiều tác động **phụ thuộc model**.

Liều thuốc: lại là che loss

masked-think — bỏ vùng suy luận rỗng khỏi loss.

response-only — che prompt *và* vùng suy luận, chỉ tính loss trên đáp án.

Cả hai **không cần trace của thầy**. Distillation cũng được — nhưng **hỏng với Olmo-3-7B**.

Quy tắc

Base có chế độ suy luận ⇒ **chủ động quyết định** khối suy luận nằm ở đâu trong loss, và thêm chỉ số "tỷ lệ trace hợp lệ" vào bộ eval (§21).

Dữ liệu tổng hợp (Synthetic Data)

Sinh, kiểm, lọc — và khi nào dữ liệu máy viết là đủ

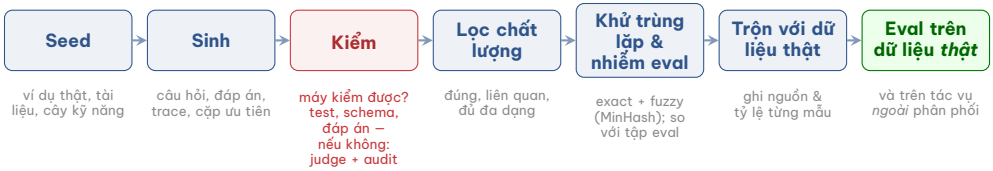
Dữ liệu tổng hợp là một *họ* phương pháp

Bắt đầu từ…	Kỹ thuật	Cơ chế một dòng	Sinh ra
Vài ví dụ mẫu	Self-Instruct, Alpaca	175 cặp mẫu → model lớn viết thêm → 52K	câu hỏi + đáp
Câu lệnh đơn giản	Evol-Instruct	viết lại từng bước cho *khó hơn*	bài khó dần
Template chat rỗng	Magpie	chỉ đưa phần đầu template, model tự "đóng vai" user	hội thoại
Persona / cây kiến thức	Persona Hub, GLAN	đổi người hỏi, đổi chủ đề để *phủ rộng*	độ phủ
Văn bản thật	back-translation	coi văn bản là đáp án, đoán câu hỏi sinh ra nó	cặp từ dữ liệu thật
Teacher biết suy luận	distill trace (s1, OpenThoughts)	ghi lại chuỗi suy luận của model lớn	chuỗi suy luận
Bộ kiểm tự động	rejection sampling (STaR)	sinh *K* đáp án, giữ cái qua test	đáp án đã kiểm

Câu hỏi đầu tiên: dữ liệu này dạy *cái gì*?

Nemotron-CC, **Cosmopedia**: dữ liệu *pre-training* (kiến thức). **SmolTalk**, **Tülu 3**: dữ liệu *SFT* (cách trả lời) — loại người fine-tune cần.

Đường ống sinh dữ liệu — và bao nhiêu mẫu sống sót



Dự án	Giữ lại	Cách lọc
AlpaGasus	9K / 52K ≈ 17%	LLM chấm điểm
Magpie	300K / 4M ≈ 7,5%	lọc nhiều tiêu chí
FROST (09/2026)	bỏ 20–30%	gradient trên dữ liệu thật
Tülu 3	bỏ *cả bộ* nếu trùng eval >2%	khử nhiễm

Lưu ý: **Không có tỷ lệ giữ "chuẩn".** 17% và 7,5% là của hai dự án khác nhau. Thói quen đúng: **ghi số dòng sau mỗi bước** — và đọc ngẫu nhiên cả những mẫu *bị loại*.

Ví dụ thật — dữ liệu của Lab 21 cũng là dữ liệu tổng hợp

Cách lab sinh 250 ticket train (`make_seed_data.py`):

```
INTENTS = {"doi_tra": [...], "van_chuyen": [...], ...}

URGENCY = {"cao": [...], "thap": [...]}

ticket = template(intent, urgency, product)

label = {"intent": intent, "urgency": urgency, ...}

random.seed(20260821) # tai lap 100%
```

Điểm mạnh: nhãn được chọn *trước*, rồi mới viết câu — đúng nếu template đúng nghĩa. Rẻ, tái lập được.

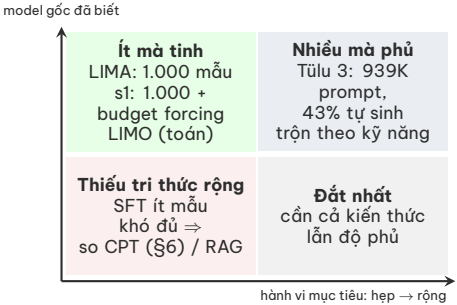
Điểm yếu: câu chữ hẹp. Khách hàng thật viết sai chính tả, trộn hai ý, gửi ảnh, nổi nóng.

Nâng cấp lên LLM sinh dữ liệu

1. Cho model lớn viết lại mỗi ticket theo 5 persona (vội, lớn tuổi, teencode…)
2. Nhãn gốc là nhãn *dự kiến*; schema chỉ kiểm cấu trúc. Kiểm intent/urgency bằng luật + judge cho ca mơ hồ; đổi nghĩa thì bỏ
3. Khử trùng lặp với `eval_target.jsonl`
4. So sánh 3 lần train: *template*, *LLM*, *trộn*

Lưu ý: Tập eval của lab sinh từ *cùng* template với tập train. Điểm cao trên đó chưa chứng minh model xử lý được ticket thật — cần một bộ *holdout* do người viết.

Ít mà tinh hay nhiều mà phủ? — tuỳ model đã biết gì



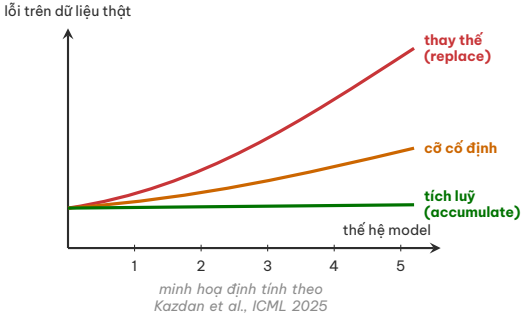
Vì sao 1.000 mẫu đủ cho LIMA/s1: model gốc *đã có* kiến thức; vài ví dụ chỉ dạy *dạng* câu trả lời. s1 còn cộng thêm compute lúc suy luận — không thể quy hết công cho con số 1.000.

Một kết quả ngược chiều: trên các tập SFT cỡ triệu mẫu, *hầu hết* phương pháp "tự chấm điểm chất lượng" không thắng nổi chọn ngẫu nhiên — **đa dạng** quan trọng hơn độ bóng của từng mẫu.

Quy tắc thực dụng

Bắt đầu nhỏ để còn đọc được dữ liệu. Tăng quy mô khi đã có *eval* và đã biết thiếu loại mẫu nào.

Model collapse — câu chuyện đúng



minh hoạ định tính theo Kazdan et al., ICML 2025

Điều đã được chứng minh (Nature, 2024): train *đệ quy* trên output của chính mình, qua nhiều thế hệ, làm mất dần **đuôi phân phối** — những trường hợp hiếm.

Điều quyết định: *thay thế* dữ liệu thật thì sụp; *tích luỹ* thêm bên cạnh dữ liệu thật thì loss ổn định.

> **Lưu ý:** **Ngộ nhận:** "một lần SFT trên dữ liệu tổng hợp đã lọc gây model collapse". Các bài báo *không* cho thấy điều đó.

Rủi ro thật ở quy mô của bạn: quên năng lực ngoài phân phối, và thừa hưởng nhiễm benchmark từ teacher (§17).

Ai cho phép bạn học từ output của họ?

Nguồn dữ liệu (teacher)	Điều khoản về việc train model khác — bản kiểm 10/2026
OpenAI (Terms, 01/2026)	cấm dùng output để phát triển model *cạnh tranh*; hợp đồng API có "Permitted Exception" (vd. fine-tune trong chính dịch vụ OpenAI)
Gemini API (03/2026)	"không được dùng dịch vụ để phát triển model cạnh tranh"
Anthropic (03/2026)	được train model *không* cạnh tranh (công cụ chuyên biệt); chatbot đa năng cần phép bằng văn bản
DeepSeek-R1 (MIT)	cho phép **rõ ràng**: "distillation for training other LLMs"
Llama 3	**cấm** dùng output để cải thiện LLM khác — điều khoản này **đã bỏ từ Llama 3.1**
Llama 3.1 trở đi (gồm Llama 4)	cho phép, nhưng tên model phân phối phải **bắt đầu bằng "Llama"**
Gemma 4, Muse Glimmer, gpt-oss (Apache 2.0)	quyền sửa đổi chung; không nêu riêng chữ "distillation"

Thói quen bắt buộc: provenance từng mẫu

Ghi **teacher, phiên bản điều khoản, ngày sinh** vào từng dòng dữ liệu — NVIDIA gắn license theo từng mẫu. "Tôi sở hữu output" ≠ "tôi được dùng nó để train mọi thứ". Pháp lý đầy đủ: §24.

Dùng model lớn để sinh dữ liệu cho model nhỏ

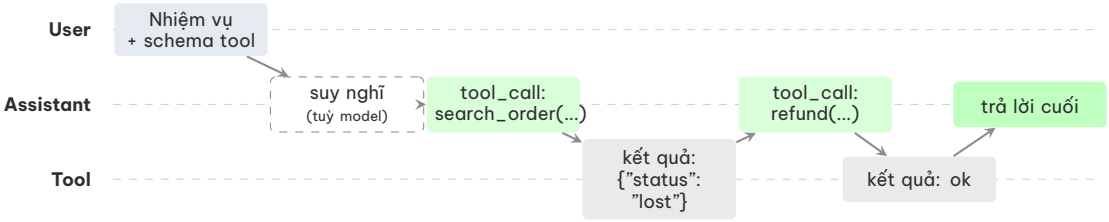
Ý tưởng: Máy viết rất rẻ; thứ đắt là *kiểm* và *chứng minh* nó giúp được.

●○○ **Cơ bản**	Model lớn viết ví dụ, bạn kiểm và lọc. Luôn giữ một bộ dữ liệu **thật** mà model chưa từng thấy để chấm điểm cuối cùng.
●●○ **Nâng cao**	Tách bước kiểm: máy kiểm được thì dùng test/schema/đáp án; không thì judge + đọc mẫu ngẫu nhiên. Ghi số dòng từng bước. So sánh ba lần train: chỉ thật, chỉ tổng hợp, trộn.
●●● **Chuyên sâu**	Lọc online theo gradient trên dữ liệu thật (FROST); on-policy distillation thay cho SFT trên output teacher (§6); theo dõi quên ngoài phân phối và provenance điều khoản của teacher.

Dữ liệu trajectory — fine-tune agent

Từ log hành động của agent tới dữ liệu huấn luyện

Trajectory là gì — một ván chơi được ghi lại từng nước



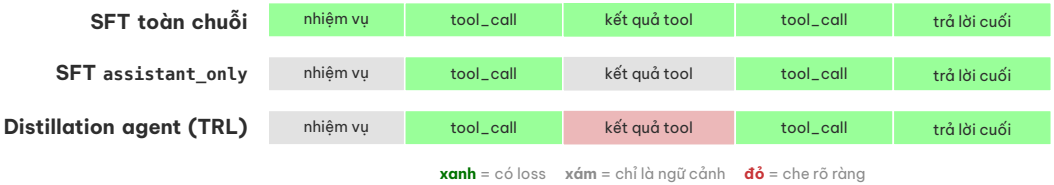
Cùng một ván, ba cách viết ra token:

- TRL: `messages` + `tools`, lượt `tool_calls` và vai `tool`
- OpenAI API: sự kiện function call có `call_id`
- Hermes / Qwen3: thẻ `<tool_call>...</tool_call>` trong văn bản

Trường suy nghĩ cũng khác nhau: `reasoning_content` (Qwen), `thinking` (Gemma).

Lưu ý: Dữ liệu ghi bằng định dạng tool của model A, train cho model B dùng định dạng khác ⇒ model học *sai giao diện*. Luôn render qua chat template của **chính** model bạn train.

Ngữ cảnh ≠ nhãn — model *thấy* gì và *học* gì



Hàng 1 còn dạy model *dự đoán* kết quả tool — một mục tiêu khác. Muốn bắt chước *hành động* agent: loss trên tool-call/trả lời, tool output giữ làm *ngữ cảnh*. Đo tác động trên hành vi thật.

"Tool output luôn bị che" là **sai** như một quy tắc chung: TRL để `assistant_only_loss=False` mặc định; chỉ đường distillation agent mới tự che kết quả tool.

Lưu ý: **Nhớ F-10 (§17):** mask lấy từ tokenizer **rỗng** khi template thiếu `{% generation %}`. Với trajectory còn nặng hơn: một mask sai che cả tool-call lẫn trả lời cuối — giải mã ngược *một trajectory đầy đủ* trước khi train.

Trajectory từ đâu ra — và đếm *đúng đơn vị*

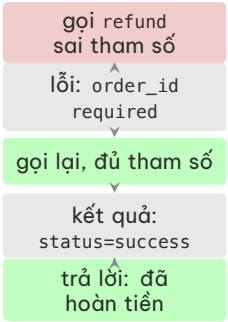
Dự án	Con số công bố	Đơn vị thật sự là gì
APIGen (2024)	3.673 API → 60K mẫu	mẫu gọi hàm, qua 3 lớp kiểm: định dạng, chạy thật, ngữ nghĩa
APIGen-MT (2025)	5.000 hội thoại	nhiều lượt, user mô phỏng; tạo task 28%→70% đạt nhờ phản hồi LLM
SWE-Gym (2024)	2.438	*task* Python có môi trường + test — chưa phải trajectory
SWE-smith (2025)	50K task → 5.000	*trajectory thành công*, tối đa 3 / task, trung bình ∼58 lượt
R2E-Gym (2025)	3.321	trajectory thành công từ 2.048 môi trường (bộ khác cho verifier)
SWE-rebench V2 (2026)	32.079	task chạy được, 20 ngôn ngữ — *chưa* có lần chạy nào

Task ≠ trajectory ≠ mẫu train

Task là đề bài. Trajectory là *một lần thử*. Mẫu train là lần thử *đã qua bộ lọc*. Ba con số này có thể chênh nhau 10 lần.

Công thức chung của các pipeline: tạo task + schema → teacher chạy thử (hoặc user mô phỏng) → **thực thi và kiểm** → giữ lần thành công. Giống hệt §18 — chỉ khác là *bộ kiểm* giờ là môi trường chạy thật.

Bước sai: giữ, che, hay bỏ? — và cái bẫy giao diện



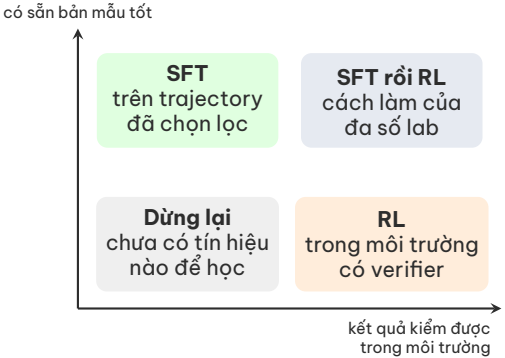
Quy tắc thực dụng: giữ bước sai *khi* nó dạy cách phục hồi đã được kiểm; đừng bắt model *bắt chước* bước sai.

Ba câu chuyện thất bại đã công bố:

- **SWE-smith:** train thẳng trên trajectory của expert *không chạy*, vì định dạng gọi tool của teacher khác của student.
- **Interface shortcut (2026):** đổi tên tool một chút, agent SFT trên trajectory tụt mạnh — nó học *giao diện*, không học *nhiệm vụ*.
- **ProCUA (06/2026):** SFT thêm trên 22,5K trajectory *người thật* (AgentNet) làm UI-TARS 7B tụt từ 26,3% xuống 8–10% trên OSWorld.

Lưu ý: Nhiều dữ liệu hơn không cứu được dữ liệu dạy sai giao diện. Trước khi train: kiểm tên tool, schema, thông báo lỗi và chính sách cắt lịch sử có *khớp với harness lúc deploy* không.

SFT trên trajectory, hay RL trong môi trường?



SFT khi hành vi lặp lại và có bản mẫu đáng tin. **RL** khi thành công phụ thuộc trạng thái thay đổi, chuỗi hành động dài, và kết quả *kiểm được*.

Môi trường giờ là dữ liệu:

- Prime Intellect Environments Hub (08/2025): môi trường = "state, action, reward"
- OpenEnv (HF, 06/2026): terminal, trình duyệt… cho agentic RL
- MiMo-V2.6 (09/2026): một lần GRPO trên >**7.000** môi trường; Granite 4.2 (08/2026): RL trong sandbox thật

Verifier là một phần của dữ liệu

> Nó định nghĩa "thành công". Verifier lỗi hoặc dễ lách ⇒ RL dạy đúng cái lỗi đó (ví dụ thật ở §20).

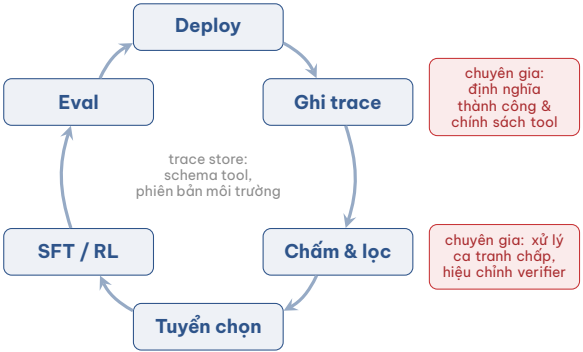
Đánh giá agent — chọn bài thi giống công việc

Benchmark	Đo cái gì	Lưu ý 10/2026
BFCL V4	gọi hàm đúng, nhiều lượt, agentic	tốt cho "gọi đúng tool, đúng tham số"
τ-bench	agent + user + tool (bán lẻ, hàng không); so trạng thái DB cuối	đo cả độ ổn định qua nhiều lần thử
τ^2-bench	*cả hai* bên đều dùng tool (viễn thông)	phối hợp với user
SWE-bench Verified	sửa issue GitHub	OpenAI **ngừng báo cáo** từ 02/2026 sau khi audit
Terminal-Bench 2	89 tác vụ dòng lệnh có test	công việc kỹ thuật dài

Một con số agent không có harness là con số không đọc được

Luôn báo kèm: **harness, schema tool, ngân sách lượt, số lần thử lại, timeout, phiên bản verifier**. Cùng một model, khác harness, có thể chênh nhiều điểm. Và đánh giá trên *chính* giao diện tool bạn sẽ deploy (§19.4).

Vòng quay dữ liệu agent — và người đứng ở đâu



Máy lo các ca *kiểm được*; người lo ba chỗ máy không tự trả lời được: **thế nào là thành công**, **ca mơ hồ hoặc rủi ro cao**, và **verifier sai ở đâu**.

Mặt trận nghiên cứu 09/2026 *(chưa thành thông lệ)*:

- AgentBrew: học từ trace MCP thô, chia điểm theo từng hành động
- Tỉa trajectory theo đồ thị phụ thuộc giữa các lượt
- TrajectoryDB: coi trajectory là một kiểu dữ liệu riêng

Fine-tune một agent từ trajectory

Ý tưởng: Agent học *hành động*, nên dữ liệu phải ghi lại hành động — đúng giao diện nó sẽ dùng.

•○○ **Cơ bản**	Một trajectory là một lần thử ghi lại từng bước: gọi tool, nhận kết quả, trả lời. Model học các bước *nó* làm, không học phần thế giới trả về.
•●○ **Nâng cao**	Render bằng template của chính model; kiểm mask bằng cách giải mã ngược; giữ bước sai làm ngữ cảnh nhưng không cho loss; đếm task, trajectory, mẫu train riêng.
●●● **Chuyên sâu**	Credit assignment theo từng hành động; RL nhiều lượt trong môi trường có verifier; kiểm thử chống "đổi tên tool"; lưu lineage của trace (schema, phiên bản môi trường).

Data engine & thị trường dữ liệu chuyên gia

Vì sao dữ liệu – không phải thuật toán – là con hào

Công cụ quen thuộc — dữ liệu và môi trường tạo khác biệt

> *"All substantive changes lie instead in the data pipeline."*
>
> — báo cáo kỹ thuật DeepSeek-V4.1-Flash (09/2026), nói về post-training của *chính họ* so với bản trước: vẫn SFT → RL → on-policy distillation.

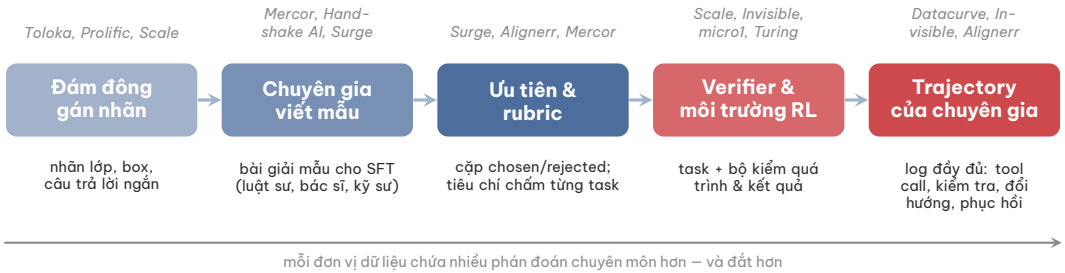
Cái họ đã đổi: tự động *sinh task* có lời giải tham chiếu và tín hiệu thưởng, *dựng môi trường* theo quy trình, rồi lọc, khử trùng lặp và hiệu chỉnh độ khó.

Release 08–09/2026	Công thức post-training
DeepSeek-V4.1-Flash	SFT → RL → OPD
MiMo-V2.6	một lần GRPO, >7.000 môi trường
Qwen3.8-Omni	chuyên gia → distill → RL chung
SEA-LION v4.8	SFT + online OPD
Muse Glimmer	SFT + OPD + RL
Granite 4.2	SFT trên trajectory + RL sandbox

Hệ quả cho bạn

Cùng bộ đồ nghề (§5–§6), thứ tự *khác nhau*. Lab lớn thắng nhờ **task, môi trường, verifier, trajectory**; dữ liệu *miền của bạn* thì họ không có.

Chuỗi giá trị dữ liệu huấn luyện 2026



Đọc theo kỹ thuật, không theo công ty: cùng một công ty thường bán ở nhiều chặng. Câu hỏi đúng là "tôi cần *loại tín hiệu* nào?" — không phải "thuê ai?".

Lab vẫn giữ quyền định nghĩa

Nhà cung cấp tìm và quản lý người; **lab quyết định thế nào là câu trả lời tốt**. InstructGPT: 40 contractor, chọn và hướng dẫn bởi chính OpenAI.

Giá dữ liệu — mỗi thẻ là một *đơn vị* khác nhau

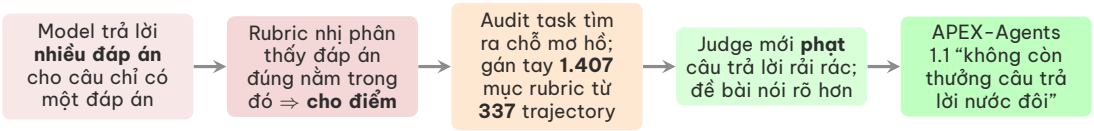
$8–12 / giờ	**$55–150 / giờ**	**≤$100 / nhãn**	**$150 / giờ**	**không công bố**
người tham gia nghiên cứu (Prolific: tối thiểu / khuyến nghị)	chuyên gia pháp lý (mức trả cho chuyên gia, Mercor)	một nhãn phức tạp (Reuters, 06/2025, nguồn ẩn danh)	lương nghiên cứu alignment nội bộ, *không* phải giá thị trường	môi trường RL, rubric, gói trajectory

Lưu ý: **Đừng vẽ chung một trục đô-la.** Giờ của người tham gia, giờ của chuyên gia, một nhãn và một môi trường là bốn đơn vị khác nhau. Không có bảng giá công khai nào cho rubric hay môi trường.

Điều có thể nói chắc: một "đơn vị dữ liệu" năm 2026 gói nhiều việc hơn — phán đoán chuyên môn, thiết kế task và rubric, dựng verifier, mô phỏng ứng dụng.

Điều chưa ai chứng minh công khai: giá toàn thị trường đã tăng bao nhiêu.

Rubric cũng bị "lách" — câu chuyện scattergunning



Vì sao chuyện này là chuyện của bạn: nếu bạn dùng rubric hay LLM judge để *lọc dữ liệu* (§18) hoặc làm *reward* cho RL (§19), model sẽ học đúng lỗ hổng của nó. Sửa prompt mà không sửa bộ chấm thì lỗ hổng vẫn còn.

Luật rõ ràng thắng tiền thưởng

Thí nghiệm 307 người gán nhãn: hướng dẫn **rõ ràng** chính xác hơn hướng dẫn mơ hồ **14 điểm**; rõ ràng + thưởng đạt 87,5%. Viết hướng dẫn tốt rẻ hơn trả thêm tiền.

Thị trường — vì sao "trung lập" cũng là sản phẩm

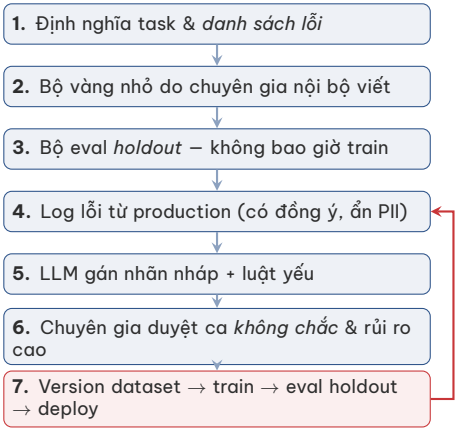
Nâng cao



Bài học kỹ thuật, không phải tin tài chính: khách hàng của nhà cung cấp dữ liệu gửi đi *prototype và ưu tiên nghiên cứu* của mình. Khi đối thủ góp vốn vào nhà cung cấp, đó là rủi ro lộ chiến lược — Reuters đưa tin khách hàng lớn *dự định* rút đi ngay sau thương vụ.

Lưu ý: Số liệu công ty tư nhân là **tự công bố hoặc nguồn ẩn danh**, chưa kiểm toán. "Mọi khách hàng đã bỏ Scale" là **phóng đại** — nguồn chỉ nói về kế hoạch và mối lo ngại.

Data engine cho một team nhỏ — không cần mua dữ liệu



> **Data-centric AI** — "Kỷ luật *kỹ thuật hoá có hệ thống* dữ liệu cần để xây hệ AI" (Andrew Ng, 2021) — một **phương pháp**, không phải một ngành hàng.

Bằng chứng có điều kiện: trong DCLM, giữ phần lớn công thức train cố định và chỉ đổi bộ lọc thì lọc bằng model thắng rõ. Model 7B cuối đạt 64% MMLU, hơn MAP-Neo **6,6 điểm** với **ít hơn 40%** compute (hai công thức khác nhau). Nhưng ở SFT cỡ triệu mẫu, chọn ngẫu nhiên gần như ngang các bộ lọc "chất lượng" (§18).

Với tiếng Việt: dữ liệu miền của bạn là thứ không lab nào có — và bộ eval nên do *người bản ngữ trong nghề* viết.

Dữ liệu là con hào — bạn đào nó thế nào?

Ý tưởng: Thuật toán ai cũng có; dữ liệu đúng miền, đúng định nghĩa "tốt", thì không.

●○○ **Cơ bản**	Mọi lab lớn dùng gần cùng một công thức; họ khác nhau ở dữ liệu. Bạn có lợi thế ở dữ liệu *miền của mình*.
●●○ **Nâng cao**	Phân biệt loại tín hiệu cần mua hay tự làm: nhãn, bài mẫu chuyên gia, cặp ưu tiên, rubric, môi trường, trajectory. Viết hướng dẫn rõ trước khi trả thêm tiền.
●●● **Chuyên sâu**	Kiểm toán bộ chấm như kiểm toán code: tìm cách model lách rubric (scattergunning), hiệu chỉnh judge với nhãn người, tách rõ dữ liệu train / eval / holdout và lineage của từng mẫu.

Đánh giá sau Fine-tune

Perplexity giảm không có nghĩa là model tốt hơn

Vì sao perplexity một mình là chưa đủ

VINUNIVERSITY

Perplexity đo cái gì

Mức "bất ngờ" của model trước văn bản trong tập của bạn.

PPL giảm nghĩa là model **giống tập dữ liệu của bạn hơn** — kể cả khi tập đó dở hoặc thiên lệch.

Một model học thuộc lòng tập train có PPL rất đẹp.

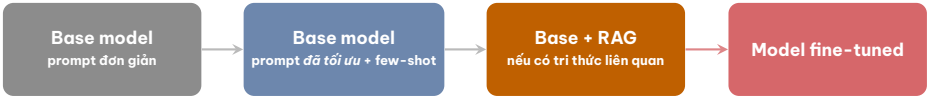
Nên đo bốn nhóm

1. **Tác vụ đích** — chính xác trên việc bạn thuê model làm
2. **Hồi quy năng lực** — benchmark tổng quát trước/sau, bắt quên thảm hoạ
3. **Tuân thủ định dạng** — tỷ lệ JSON/schema hợp lệ
4. **Vận hành** — độ trễ, token/giây, chi phí mỗi yêu cầu

Cổng hồi quy

Trước khi train, **đóng băng** tập eval và ghi điểm base model. Sau khi train, chạy lại đúng tập đó. Nhóm (2) tụt quá ngưỡng ⇒ **không deploy**, bất kể nhóm (1) đẹp đến đâu. Eval đầy đủ: Ngày 14.

Ba baseline phải so — RAG khi bài toán cần



Lưu ý: So bản fine-tune với **base model + prompt tệ** là cách tự lừa mình phổ biến nhất. Gần như lần nào bạn cũng "thắng" — và không học được gì.

Câu hỏi kết luận

Bản fine-tune có thắng **base model đã được prompt tử tế** một khoảng đủ lớn để bù cho chi phí huấn luyện, phục vụ và bảo trì nó không?

Thất bại thường gặp & Debug

Triệu chứng → nguyên nhân gốc — những lỗi loss không hề báo cho bạn

Bảng tra triệu chứng

Triệu chứng	Nguyên nhân hay gặp nhất	Kiểm tra
Model **không bao giờ dừng** sinh	EOS lúc train ≠ EOS lúc suy luận	So `eos_token` với token template phát r
Model *viết tiếp* câu hỏi của bạn	Loss tính cả trên prompt	Bật `completion_only_loss` (§17.2)
Đầu ra lặp / vô nghĩa	Sai chat template hoặc special token	Giải mã ngược một mẫu đã tokenize
Loss phẳng ngay từ step 0	Mask sai, hoặc LR thấp một bậc	Kiểm tra mask; nhân LR lên 10
Điểm eval đẹp, production tệ	Trôi phân phối / nhiễm benchmark	Eval trên đầu vào người dùng thật
Giỏi miền mới, dốt mọi thứ khác	Quên thảm hoạ	Thêm 1–5% replay (§6.3)
Gai loss rồi NaN	LR quá cao / tràn số	Gradient clipping; kiểm tra dữ liệu
Mất suy luận, accuracy vẫn tăng	Template nuốt khối `<think>`	In chuỗi *sau* `apply_chat_template` (§17.

Lưu ý: Không lỗi nào ở trên làm training **dừng lại**. Tất cả đều chạy đến hết, in ra một đường loss trông hợp lý, rồi giao cho bạn một model hỏng. Đây là lý do phải kiểm tra *trước* khi train.

Ca bệnh điển hình — lệch chat template

Sự cố có thật

Một cấu hình Qwen3 base đặt `eos_token` thành `<|endoftext|>`, trong khi chat template vẫn phát ra `<|im_end|>`.

Hai token khác nhau cho cùng một khái niệm "kết thúc lượt".

IFEval: 67,84 → 41,96.

Không có thông báo lỗi nào. Training chạy bình thường.

Quy tắc vàng

Áp **đúng một** chat template lúc huấn luyện và lúc suy luận. Nếu khác nhau, model của bạn kỳ vọng một định dạng mà *không ai* gửi tới.

Danh sách kiểm tra 3 dòng

1. `tokenizer.eos_token` là gì?
2. Template kết thúc lượt bằng token nào?
3. Hai cái đó có **giống nhau** không?

Khi fine-tune một **base** model bằng dữ liệu hội thoại, bạn thường *phải* đổi `eos_token` cho khớp template — base model chưa có khái niệm "hết

Merge, Deploy & Multi-LoRA Serving

Quyết định lúc huấn luyện quyết định kinh tế lúc phục vụ

Hai đường triển khai — gộp hay giữ riêng

Gộp (merge)

$W = W_0 + \frac{\alpha}{r} BA$ — adapter tan vào trọng số gốc.

Được: đồ thị suy luận *giống hệt* base model $\Rightarrow$ **0% overhead**; xuất được GGUF/AWQ; chạy được ở mọi backend.

Mất: một model đầy đủ cho *mỗi* biến thể — *n* khách hàng là *n* bản sao.

Giữ riêng (multi-LoRA)

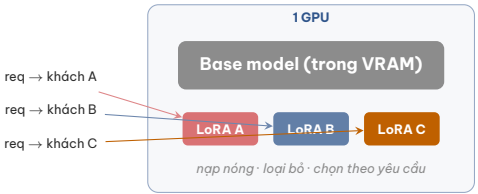
Một base model trong VRAM + nhiều adapter nạp theo yêu cầu.

Được: *n* khách hàng trên **một** GPU; thêm/bớt adapter nóng, không khởi động lại.

Mất: một ít overhead mỗi yêu cầu; backend phải hỗ trợ.

Lưu ý: DoRA: vector độ lớn phải được gộp đúng cách — cần PEFT $\geq$ 0.10. Với phiên bản cũ hơn, model sau khi merge *không* bằng model lúc train, và bạn chỉ phát hiện ra khi eval sau merge tụt điểm.

Multi-LoRA — kinh tế học của một base, nhiều adapter



- vLLM cho phép cập nhật adapter lúc chạy qua `VLLM_ALLOW_RUNTIME_LORA_UPDATING` cùng endpoint nạp/gỡ
- **S-LoRA** báo cáo **tới 4**× thông lượng so với hỗ trợ LoRA ngây thơ, và nhiều bậc độ lớn về số adapter đồng thời, nhờ batching không đồng nhất + kernel CUDA riêng
- Cô lập khách hàng nằm ở tầng định tuyến, không ở tầng model

Vì sao đây là lập luận kinh doanh

Fine-tune riêng cho từng khách hàng thường bất khả thi vì *chi phí phục vụ*, không phải chi phí huấn luyện. Multi-LoRA xoá đúng khoản đó.

Chi tiết stack phục vụ: **Ngày 20**.

Engine nào phục vụ adapter — và khi bạn có *nhiều* adapter

VINUNIVERSITY

Hai engine, hai điểm mạnh

Engine	Mạnh ở
vLLM	multi-LoRA trưởng thành: hot-reload, xoay vòng adapter; nạp trọng số adapter *chồng lấn* với tính toán
SGLang	multi-LoRA là công dân hạng nhất: các request **khác adapter** dùng chung một batch; RadixAttention cache tiền tố

Quy tắc trong tài liệu: 5–10 adapter → cái nào cũng được; **50+** adapter lưu lượng không đồng nhất → lịch biểu của SGLang; tiền tố dùng chung nhiều (chat, RAG) → SGLang. Throughput cụ thể: **chưa có nguồn sơ cấp** — đừng trích.

Gộp *nhiều* adapter là bài toán khác

Cộng tuyến tính trọng số nhiều adapter thường gây **nhiễu phá hoại**: cập nhật của tác vụ này triệt tiêu tác vụ kia.

Task Arithmetic — trung bình các delta

TIES — cắt tỉa → bầu dấu → gộp rời

DARE — reset ngẫu nhiên phần dư, co giãn phần còn lại

Kỷ luật bắt buộc

Chấm checkpoint đã gộp **cả với base lẫn với từng chuyên gia nguồn** — rồi mới đưa ra sản xuất.

License, Pháp lý & Dữ liệu

Bản fine-tune của bạn thừa kế nhiều thứ hơn bạn nghĩ

License đi theo bản fine-tune của bạn

Họ model	License	Hệ quả khi bạn fine-tune
Qwen3 (phần lớn)	Apache 2.0	Fine-tune, thương mại hoá, phát hành lại — chỉ cần ghi công
Gemma 4	Apache 2.0 *(từ 04/2026)*	Như trên; vẫn còn Prohibited Use Policy riêng
Muse Glimmer (Meta)	**Apache 2.0** *(08/2026)*	Chính Meta nay cũng phát hành giấy phép dễ dãi
DeepSeek-V4, GLM-5	MIT	Ràng buộc lỏng nhất trong nhóm
Kimi K3	License riêng	**Phải đọc toàn văn** trước khi thương mại hoá
Llama	Community License	Bản fine-tune **là** Llama derivative — xem cảnh báo dưới

Điều khoản âm thầm phá pipeline

License Llama cấm dùng **đầu ra hoặc bản fine-tune của Llama để huấn luyện / chưng cất một model *không phải* Llama**.

Nghĩa là: sinh dữ liệu tổng hợp bằng Llama rồi train Qwen là **vi phạm**.

Hạ nguồn cũng thừa kế AUP, ngưỡng 700M MAU và điều khoản này.

Câu hỏi cần trả lời trước khi chọn base

1. Tôi được phép thương mại hoá không?
2. Tôi có phải phát hành lại dưới cùng license không?
3. Đầu ra của model này có được dùng để train model khác không?
4. Có ngưỡng người dùng kích hoạt điều khoản mới không?

Chọn base model là quyết định **pháp lý** ngang với kỹ thuật. Và bản đồ này *đổi theo quý* — kiểm tra lại license mỗi lần khởi động dự án.

PDPL — và vì sao fine-tuning là ca khó nhất

Luật 91/2025/QH15 — Luật Bảo vệ dữ liệu cá nhân — Quốc hội thông qua 26/06/2025, **hiệu lực 01/01/2026**. Nghị định 356 thay thế Nghị định 13/2023 làm văn bản hướng dẫn từ cùng ngày.

Nghĩa vụ liên quan trực tiếp đến AI:

- Giới hạn thu thập theo mục đích đã xác định
- Mã hoá & kiểm soát truy cập
- **Ẩn danh hoá / phi định danh** khi khả thi
- Minh bạch về logic ra quyết định tự động, có cơ chế từ chối

Điểm khác biệt cốt lõi

RAG lưu dữ liệu trong **index** — xoá một bản ghi là chuyện của một câu lệnh.

Fine-tuning **nung dữ liệu vào trọng số**. Một yêu cầu xoá dữ liệu cá nhân không có cách xử lý rẻ — câu trả lời trung thực là **huấn luyện lại**.

Hệ quả thiết kế

Phi định danh phải là quyết định ở **khâu dựng tập dữ liệu**, không phải việc vá sau. Sau khi trọng số đã học, đã muộn.

Demo & Thực hành

Fine-tune một model tiếng Việt — và chứng minh nó thật sự tốt hơn

Bốn thứ hỏng mà loss không hề báo

LIVE DEMO

1. **Mask sai:** giải mã ngược `labels` — mask lấy từ tokenizer giám sát **0 token** trên template không có `{% generation %}` — TRL báo lỗi, pipeline tự viết thì im lặng
2. **Lệch template:** train một dạng prompt, chấm một dạng khác — loss đẹp, điểm đích **0,000**
3. **Thang learning rate:** cùng dữ liệu, LR full-FT so với LR $\times 10$ — hai đường loss khác hẳn
4. **Xếp hạng bằng loss:** run `attn_only` có loss huấn luyện *thấp hơn* run đúng — và vẫn là cấu hình tệ hơn

LAB #21

Mục tiêu: Fine-tune một open-weight model bằng LoRA — và chứng minh nó thắng được base model đã prompt tử tế

Deliverable: Adapter LoRA + báo cáo đối chiếu **ba baseline**, **bốn nhóm điểm**, kèm ba run đối chứng cùng ngân sách step

Thời gian: ≈2 giờ trên Colab Free T4

Lab 21 — năm notebook, và cái mà mỗi cái ép bạn chứng minh

NB	Tên	Phải chứng minh điều gì	T4 (đo thật)
1	data & mask	phần được tính loss **đúng bằng** câu trả lời — giải mã ngược	≈25 giây
2	baselines	mốc được **đóng băng TRƯỚC** khi train: (a) prompt ngây thơ, (b) prompt đã tối ưu	17–23 ph
3	train correct	cấu hình vùng-không-hối-tiếc, in `layer_types` của chính model	15–25 ph
4	misconfig autopsy	3 run đối chứng **cùng số step**: `attn_only` · `wrong_lr` · `qlora`	45–60 ph
5	evaluate & verdict	4 nhóm + **cổng hồi quy** — và một phán quyết có thể là *THẤT BẠI*	≈21 ph

Cấu hình `correct` — vì sao khác §14

`all-linear` · r=16 · α=2r · LR ≈10× · batch <32 · `chunked_nll`

LoRA bf16/fp16, KHÔNG phải QLoRA — 4-bit là một *run đối chứng*. `packing` tắt; `padding_free` không dùng được (§15.1).

Lưu ý: Bốn nhóm đều có thang **khách quan** — không LLM judge, nên không có "điểm cho không": **target** · **regression** (15 câu phổ thông) · **format** (JSON đủ 4 khoá) · **latency**.

Điểm nằm ở chỗ bạn *biết*, không phải ở chỗ bạn *thắng*

Baseline (b) là một cái bar thật

Bản fine-tune phải thắng **base model đã được prompt tử tế** — không phải thắng base model bị prompt cẩu thả. Rất nhiều báo cáo "fine-tune thành công" chỉ thắng được (a).

`make verify` kiểm tra phép so sánh bạn được chấm là **công bằng**: checksum eval, ngân sách step dùng chung, artefact đủ.

Hai cần gạt khi bị bó thời gian

`EVAL_LIMIT=8` → rút ngắn phần **sinh**

`EPOCHS=1` → nửa số **step**

Đo thật: cả hai bật, NB1+2+3+5 hết **17 phút**. Để **mặc định** khi nộp bài.

`EPOCHS` áp cho **cả** NB3 lẫn NB4 — không chỉnh được một nửa. Ba run đối chứng chỉ có nghĩa khi chạy đúng bằng số step của `correct`.

Phát hiện đắt giá nhất của lab này

Một lần chạy thật đã cho **cả bốn** adapter điểm target = 0,000 — trong khi loss hoàn toàn khoẻ mạnh. Nguyên nhân: train một dạng prompt, chấm một dạng khác. **Một mask loss đã kiểm chứng không bảo đảm một prompt đúng** — đó là hai trục khác nhau.

Tổng kết — Key Takeaways

Những ý chính cần nhớ trước khi sang bài tiếp theo

1. **Quyết định:** leo bậc thang baseline; chỉ đổi trọng số khi đo được khoảng cách
2. **Model đến từ đâu:** SFT, RL, distillation là đồ nghề chung; khác biệt nằm ở *dữ liệu và môi trường*
3. **Lõi:** LoRA đủ dung lượng, phủ mọi lớp, dò đúng LR thì gần full FT — xác nhận bằng eval
4. **Dữ liệu:** sinh thì rẻ, *kiểm* mới đắt — verifier, provenance, eval trên dữ liệu thật
5. **Ra sản phẩm:** bộ hồi quy & an toàn; adapter phụ thuộc base, license, endpoint

Tiếp theo & Bài tập

Bài tiếp theo

Ngày 22: DPO, ORPO & Alignment

"Hôm nay ta dạy model cách trả lời. Ngày mai: dạy nó trả lời tốt hơn theo sở thích con người — với RL, rank-1 adapter đã đủ; DPO có giống vậy không?"

Bài tập về nhà

- Hoàn thành Lab 21 — kèm bảng ba baseline
- Đọc: Rafailov et al., "Direct Preference Optimization" (2023)
- Đọc (khuyến nghị): Thinking Machines Lab, "LoRA Without Regret" (2025)

Hỏi & Đáp

Use case của bạn thật sự cần fine-tune, hay chỉ cần một prompt tử tế hơn? Hãy mang một ví dụ cụ thể ra mổ xẻ.



Cảm ơn!

AICB-P2T3 · Ngày 21 · Fine-tuning LLMs
— Từ Full Fine-tune đến LoRA/QLoRA

github.com/vinuni-aicb

Liên hệ: instructor@vinuni.edu.vn